

University of Stuttgart
Germany
Institute of Industrial Automation and
Software Engineering

Risk-Annotated Spatial Scene Graph for Dy- namic Risk As- sessment in Autonomous Systems

Master Thesis TBD

Submitted at the University of Stuttgart by
Fabin Fize! Sha

Electrical Engineering

Examiner: Prof. Dr.-Ing. Andrey Morozov

Supervisor: Joachim Grimstad, M.Sc.

**External
Supervisor:** ()

30 October 2026



Abstract

Conventional risk assessment mechanisms are limited when the control strategy or operating environment of an autonomous system changes after deployment. This limitation is particularly relevant for systems operating in dynamic environments or modified through adaptive software components and over-the-air updates.

This thesis extends the framework for online risk-annotated scene graph generation presented by André Schiel in the master's thesis *Online Risk-Annotated Scene Graphs for Autonomous Systems* [1] into a spatial scene graph that supports dynamic risk assessment in near real time. The scene graph represents objects together with their spatial context, semantic labels and risk-relevant object descriptions generated by vision-language models, and an uncertainty estimate of whether each object is still at its last observed position. The framework is architected as a modular ROS 2 system, which supports portability and integration with robotic platforms, and it builds on the Hydra spatial perception system to support large environments.

Development and optimisation were carried out on an NVIDIA Jetson AGX Orin using the office scene of the simulated uHumans2 dataset, and the system was subsequently ported to an NVIDIA Jetson AGX Thor. Qwen3.5-4B combined with an optimised prompt was selected for object labelling, reaching an accuracy of 88 % on 50 manually verified object crops. Additional tests on the uHumans2 apartment scene and on multiple recordings from the office scene of the real-world OpenLORIS-Scene dataset demonstrate portability across environments and across simulated and real sensor data. On the Jetson AGX Thor, the pipeline processes [XX] frames per second on [dataset and configuration]. This near-real-time rate is an improvement over the baseline framework and makes the risk-annotated scene graph available as environmental context during system operation.

Keywords: dynamic risk assessment, risk-annotated scene graph, ROS 2, Hydra, uncertainty-aware object representation, vision-language model, edge AI

Contents

Abstract	ii
List of Figures	v
List of Tables	vi
1. Introduction	1
1.1. Motivation	1
1.2. Relevance	2
1.3. Goal of the Thesis	2
1.4. Contributions	3
2. Theoretical Background	4
2.1. Autonomous Systems and Safety	4
2.1.1. Human-Robot Collaboration	5
2.1.2. Conventional Risk Assessment and Risk Reduction	6
2.1.3. Existing Safety Measures in HRC	6
2.1.4. Impact of Artificial Intelligence	8
2.2. Existing Work on Spatial Scene Graphs	9
2.2.1. 3D Scene Graph by Armeni et al.	9
2.2.2. SceneGraphFusion	10
2.2.3. ConceptGraphs	11
2.2.4. OpenGraph	13
2.3. Robot Operating System	14
2.3.1. ROS as Robotic Middleware	14
2.3.2. ROS Communication Mechanisms	15
2.3.3. ROS 1 and ROS 2	15
2.4. Scene Graphs	16
2.5. Hydra-Based Spatial Scene Graph Generation	18
2.5.1. Input Data and Processing Overview	19
2.5.2. Layered 3D Scene Graph Representation	19
2.5.3. Incremental Construction of Spatial Layers	20
2.5.4. Loop Closure, Optimization, and Real-Time Architecture	20
2.5.5. Relevance to the Proposed Framework	21

Contents

2.6.	Risk-Annotated Scene Graph Framework Baseline	21
2.6.1.	Information Flow in the Baseline Framework	22
2.6.2.	FastAPI	23
2.6.3.	Scene Graph Construction and Risk Annotation	23
2.6.4.	Limitations of the Baseline Framework	24
2.7.	Lightweight Promptable Segmentation with NanoSAM	25
2.7.1.	Architecture of the Segment Anything Model	26
2.7.2.	Knowledge Distillation and MobileSAM	27
2.7.3.	NanoSAM	27
2.7.4.	Limitations	28
2.8.	Multi-Object Tracking and Data Association	28
2.8.1.	The Data Association Problem	29
2.8.2.	Object-Level SLAM with Fusion++	29
2.8.3.	Limitations of Fusion++	30
2.8.4.	Relevance for Persistent Scene Representations	31
2.9.	Visual Retrieval and Vision-Language Models	31
2.9.1.	Retrieval-Augmented Perception	31
2.9.2.	Vision-Language Models	33
2.9.3.	The Qwen Vision-Language Model Family	34
2.10.	Embedded Computing Hardware: NVIDIA Jetson Platforms	35
2.10.1.	NVIDIA Jetson AGX Orin Developer Kit	35
2.10.2.	NVIDIA Jetson AGX Thor Developer Kit	36
2.11.	Scene Graphs for Downstream Safety Reasoning	36
3.	Methodology	38
3.1.	Baseline System and Proposed Extensions	38
3.2.	Hardware Setup and Input Data Preparation	38
3.3.	Data Preprocessing	40
3.4.	Pre-Processing	41
4.	Experiments	42
5.	Conclusions	43
	Bibliography	44
A.	Example appendix chapter	50

List of Figures

2.1.	Main challenges in HRC[19].	5
2.2.	Illustrative representation of safety zones in a collaborative robot workspace.	7
2.3.	Overview of the SceneGraphFusion pipeline, showing RGB-D input, incremental geometric segmentation, point-cloud segments as graph nodes, feature extraction, graph neural network-based relationship prediction, and temporal fusion into an incremental 3D scene graph. Reproduced from [22].	10
2.4.	Overview of the ConceptGraphs pipeline, showing open-vocabulary detection and segmentation from posed RGB-D observations, multi-view object association, object fusion, construction of a 3D scene graph, and downstream applications such as segmentation, grounding, navigation, and manipulation. Reproduced from [8].	12
2.5.	Overview of the OpenGraph pipeline, showing caption-enhanced object comprehension from RGB images, object-centric 3D map construction using posed point clouds, and hierarchical scene representation formation for large-scale outdoor environments. Reproduced from [23].	13
2.6.	Scene graph example. Reproduced from [26].	17
2.7.	Example of a layered 3D scene graph generated by Hydra, showing abstraction levels from metric-semantic 3D mesh to objects and agents, places, rooms, and buildings. Reproduced from [7].	18
2.8.	Information flow of the existing risk-annotated scene graph framework. Adapted from [1].	22
2.9.	Overview of the Segment Anything Model (SAM) architecture. The model consists of an image encoder, a prompt encoder, and a lightweight mask decoder that produces segmentation masks conditioned on input prompts. Reproduced from Kirillov et al. [30].	26
2.10.	Principle of visual retrieval: the crop embedding is compared with stored references, and the nearest label is returned if its distance lies below the threshold δ	32
2.11.	SafetyDetect pipeline showing the agent point of view, scene graph construction, prompt formation, and LLM-based anomaly classification. Reproduced from [9].	37

List of Tables

2.1. Latency on Jetson AGX Orin and mean IoU on COCO 2017 validation with ground-truth box prompts, as reported in [33].	28
2.2. Release history of the open-weight Qwen vision-language models.	34

1. Introduction

1.1. Motivation

Robotic systems are increasingly moving out of fenced industrial cells and into shared, dynamic environments such as warehouses, hospitals, offices, and public spaces. According to the International Federation of Robotics, almost 200,000 professional service robots were sold in 2024, an increase of 9% over the previous year [2]. The largest share of these, 102,900 units, were built for transportation and logistics and consist mainly of mobile robots that move goods through their surroundings [2]. At the same time, the global operational stock of industrial robots reached 4.66 million units in 2024 [3]. Together, these figures show that robots are becoming a common part of environments that are also used by people.

Alongside this growth, robotic systems are increasingly shaped by artificial intelligence. The International Federation of Robotics identifies physical, analytic, and generative AI as key technological trends in robotics [3]. AI-based components allow robots to perceive their surroundings, interpret scenes, and adapt their behaviour. At the same time, they make robot behaviour harder to predict fully during design-time safety analysis [4].

This development creates new challenges for safety and risk assessment. Conventional risk assessment is typically carried out before deployment and relies on assumptions about the system design, intended use, operating environment, and known hazards [5]. These methods remain essential, but they reach their limits when mobile robots operate in changing environments, receive over-the-air software updates, or use adaptive AI-based components. In such cases, the situation the robot actually faces may differ from the one assumed during the original assessment. For this reason, automated and continuous risk assessment has been proposed, in which the assessment is updated while the system is running [6].

Such dynamic risk assessment needs a structured description of the current environment. It must know which objects are present, where they are, how they relate to each other, and which of them may be relevant to risk. A spatial risk-annotated scene graph is a suitable way to provide this information, as it combines perceived objects, their spatial context, and risk-related properties in a single machine-readable representation. In this thesis, the term *risk-annotated scene graph* refers to a spatial scene graph whose object nodes are enriched with risk-relevant semantic and uncertainty information, including object labels, confidence values, mobility information, presence likelihood, and descriptive object properties. These annotations provide structured environmental context for downstream dynamic risk assessment rather than representing a complete risk assessment by themselves.

1.2. Relevance

Scene graphs have become an established way to represent robotic environments in a structured form. Hydra [7] builds a layered 3D scene graph of objects, places, and rooms in real time and keeps it consistent through loop closure. However, Hydra does not identify objects itself. Its object layer depends on per-pixel semantic labels, which come from ground-truth annotations in simulation or from a segmentation network trained on a fixed set of classes [7]. ConceptGraphs [8] removes this restriction by identifying objects with open-vocabulary foundation models, but it applies these models to every observation and represents objects through free-form descriptions and features rather than bounded, confidence-weighted information.

At the same time, work on safety reasoning shows that scene graphs are a useful input for detecting hazardous situations. SafetyDetect [9], for example, uses object relations from a scene graph to let a language model classify situations as normal or dangerous. The authors note that their approach is limited mainly by how the scene graph is created, since risk assessment can only consider what the scene graph actually contains. The quality of the underlying scene graph is therefore a key factor for any risk assessment built on top of it.

The existing risk-annotated scene graph framework [1] brings these two directions together by adding risk-related information to a scene graph generated from RGB-D data. It is, however, limited to local, frame-based scene understanding and is exposed through a service interface that is not designed for robotic middleware. It also has no mechanism for keeping objects consistent across a larger environment, and it provides only basic handling of perception uncertainty. There is therefore a need for a risk-annotated scene graph that combines the spatial consistency of systems like Hydra with flexible object identification and risk-relevant object information, and that is designed for integration into a mobile robotic system during operation.

1.3. Goal of the Thesis

The goal of this thesis is to extend the existing risk-annotated scene graph framework [1] into a 3D spatial scene graph that provides up-to-date environmental context for dynamic risk assessment. The scene graph should describe the objects in the environment, their spatial context, and their risk-relevant properties. The work focuses on four aspects: portability, spatial scalability, uncertainty awareness, and near-real-time operation.

The first goal is to integrate the baseline framework into ROS 2 [10], so that it can interact directly with robotic sensors, localisation, and mapping components. The second goal is to ground the scene graph in a spatial map, so that objects observed at different times and locations share a common reference frame and remain spatially consistent when areas are revisited. Object identification should support open-vocabulary semantic perception, removing the dependence on ground-truth semantic labels or a fixed set of object classes.

1. Introduction

The third goal is to represent uncertainty associated with object identification, mobility, and continued presence in the environment. The fourth goal is to support near-real-time operation on embedded robotic hardware. Finally, the framework is evaluated on the simulated uHumans2 dataset [11] and the real-world OpenLORIS-Scene dataset [12], focusing on object tracking and labelling, scene graph consistency, and runtime behaviour.

1.4. Contributions

This thesis makes the following contributions:

1. **A ROS 2-based risk-annotated scene graph framework.** The baseline framework is adapted for ROS 2, enabling direct integration with the sensors and software components of a robotic system. This provides a foundation for using risk-relevant scene representations within a broader robotic software architecture.
2. **A spatial scene graph with open-vocabulary object identification.** The framework extends Hydra [7] with flexible object identification beyond fixed semantic classes. The resulting 3D scene graph combines spatial, semantic, and mobility information while maintaining a consistent representation of the environment over time.
3. **A persistent object representation with uncertainty information.** Objects are represented together with confidence information for their semantic identity, mobility, and continued presence in the environment. This allows the scene graph to express both what is currently believed about an object and the uncertainty associated with that information.
4. **A design for near-real-time operation on embedded robotic hardware.** The framework is designed to maintain spatial scene understanding while incorporating computationally demanding semantic information. This allows the scene representation to remain suitable for applications in which environmental information must be updated continuously during robot operation.
5. **An experimental evaluation on simulated and real-world data.** The extended framework is evaluated using the simulated uHumans2 dataset [11] and the real-world OpenLORIS-Scene dataset [12]. The evaluation examines object tracking and labelling, spatial scene graph consistency, and runtime behaviour to identify both the capabilities and limitations of the proposed approach.

2. Theoretical Background

2.1. Autonomous Systems and Safety

Autonomous systems (AS) can be understood as systems that are capable of making decisions independently from direct human control [13]. In robotic systems, this autonomy is typically achieved through a combination of sensing, perception, planning, decision-making, and actuation. For example, an autonomous mobile robot may perceive obstacles, select an alternative path, and continue its task without requiring continuous human input. This ability allows autonomous robots to operate more flexibly in dynamic and partially unknown environments. However, it also makes safety assurance more complex, because the system behaviour is no longer determined only by predefined commands, but also by runtime perception, environmental conditions, and decision-making processes.

Safety is a fundamental requirement in AS because robots interact with humans, physical objects, infrastructure, and the surrounding environment. In dependability theory, safety is commonly defined as the absence of catastrophic consequences for users and the environment [14]. In the context of autonomous robotic systems, safety therefore refers not only to whether the robot performs its intended task, but also to whether it performs the task without creating unacceptable risk. This distinction is important because a robot may be functionally successful while still being unsafe. For example, a robot arm in a shared workspace may complete a pick-and-place task correctly, but the system cannot be considered safe if its motion exposes a nearby human operator to an unacceptable collision risk.

From a risk assessment perspective, safety is closely related to the identification of hazards, the estimation of risk, and the implementation of suitable risk reduction measures [5]. In robotic applications, this requires considering both the severity of possible harm and the likelihood that a hazardous situation may occur. Robot-specific safety standards further define requirements and protective measures for industrial and collaborative robotic systems [15]–[17]. However, safety in AS is strongly dependent on operational context. A motion, speed, or decision that is safe in one situation may become hazardous when a human enters the workspace, when an object is misplaced, or when perception uncertainty increases. Therefore, safety in autonomous robotic systems must be understood as a property that depends on both the technical system and the changing environment in which it operates.

2.1.1. Human-Robot Collaboration

Industry 5.0 is shifting industrial development toward a more human-centric, sustainable, and resilient model, while continuing to support productivity and worker well-being [18]. Within this perspective, Human-Robot Collaboration (HRC) becomes an important requirement for using robotic technologies to support and empower workers rather than simply replacing human labour [18]. Collaborative robotic systems can assist humans in repetitive, strenuous, or hazardous tasks, thereby improving workplace flexibility, ergonomics, and the distribution of work between humans and machines. In industrial settings, HRC has also been discussed as a response to skill gaps, skilled-worker retention, and the need to make manufacturing work more attractive to younger generations [19]. This makes HRC a relevant foundation for future autonomous systems, where robots increasingly need to interpret human presence, environmental changes, and task-specific context during operation.

However, HRC also introduces important challenges in industrial settings. Industrial HRC must address three closely related aspects: ensuring human safety, maintaining trust in automation, and preserving productivity [19]. Among these, human safety remains a primary concern because close physical interaction increases the possibility of unintended human-robot collisions, unexpected robot motion, or interruptions in the robot's behaviour. Such situations can increase the risk of injury and may negatively affect the human operator's trust in the system. At the same time, safety measures must be balanced with productivity requirements, since overly restrictive measures such as excessive speed reductions, large separation distances, or frequent safety stops can reduce system efficiency and interrupt the intended collaborative workflow [19]. The relationship between safety, trust in automation, and productivity is illustrated in Figure 2.1. This trade-off shows that HRC cannot be addressed only as a productivity or automation problem; it also requires systematic risk assessment methods that can identify hazardous situations and define suitable risk reduction measures.

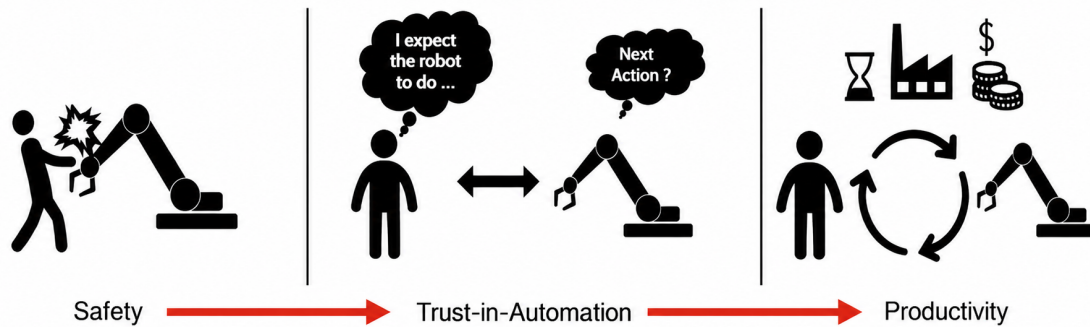


Figure 2.1.: Main challenges in HRC[19].

2. Theoretical Background

2.1.2. Conventional Risk Assessment and Risk Reduction

Conventional risk assessment in robotic systems is generally performed before deployment and is based on the intended use, foreseeable misuse, system limits, operating environment, and known hazards of the system. According to ISO 12100, the risk assessment process includes hazard identification, risk estimation, and risk evaluation, followed by the selection of suitable risk reduction measures [5]. In the context of industrial and collaborative robots, this process is supported by robot-specific safety standards that define requirements for robot systems, collaborative applications, and protective measures [15]–[17].

Risk reduction is usually implemented through a combination of inherently safe design, technical protective measures, and information for use. In HRC, this can include limiting robot speed and force, defining safe separation distances, using protective stops, and applying physical or electronic safeguards around the robot workspace. These measures are essential because they provide a systematic way to reduce the likelihood or severity of hazardous human-robot interactions before the system is put into operation.

However, conventional risk assessment is mainly based on assumptions made during design and deployment. These assumptions include expected human behaviour, known object configurations, defined workspaces, and predefined task conditions. As a result, conventional risk assessment is effective for identifying known and reasonably foreseeable hazards, but it becomes limited when the operating context changes significantly during runtime. This limitation is especially relevant for autonomous robotic systems, where the robot may operate in dynamic environments and make decisions based on sensor data and perception results.

2.1.3. Existing Safety Measures in HRC

In industrial HRC, risk reduction strategies are commonly implemented through two main approaches: the use of collaborative robots and the use of physical or electronic safeguarding measures [19]. Collaborative robots, also referred to as cobots, are designed to operate in direct cooperation with humans within a defined shared workspace. Their safety is supported through hardware and software features such as reduced momentum, force sensing, back-drivable actuators, and protective stop functions that interrupt robot motion when unexpected contact or resistance is detected.

The safeguarding-based approach focuses on monitoring the robot workspace so that robot motion can be adapted or stopped before hazardous human-robot collisions occur. In practice, this is achieved using physical and electronic safeguarding devices that define protected zones around the robot work cell. Examples include safeguarded perimeter door switches, light curtains, pressure-sensitive floors, 2-D scanning LiDAR, and 3-D safety vision systems. These mechanisms are important for reducing the risk of human injury, but they often rely on predefined safety zones, fixed distance thresholds, or conservative stopping rules [19].

2. Theoretical Background

A common method used for workspace monitoring is Speed and Separation Monitoring (SSM). In SSM, the distance between the human and the robot is monitored continuously, and the robot speed is reduced or stopped when the separation distance becomes too small. In a static SSM approach, the monitored zones are usually predefined before operation. For example, a robot may operate at full speed when no human is detected in the outer zone, slow down when a human enters a warning zone, and stop when the human enters a protective stop zone[19]. This approach is simple and practical, but it can be conservative because the safety zones are not always adapted to the actual robot motion, human motion, task state, or sensor uncertainty.



Figure 2.2.: Illustrative representation of safety zones in a collaborative robot workspace.

Dynamic SSM addresses this limitation by adapting the protective separation distance or the robot speed according to the current operating situation. Instead of relying only on fixed zones, dynamic SSM considers factors such as the actual distance between the human and the robot, robot velocity, human motion, stopping time, and uncertainty in the sensed position of obstacles or operators. Marvel discusses SSM performance in terms of both safety and productivity, representing collision potential through separation metrics and sensor uncertainty [20]. This shows that SSM is not only a binary stop-or-run mechanism, but also a method whose effectiveness depends on how accurately the system evaluates separation, uncertainty, and the productivity cost of safety actions.

Although dynamic SSM provides a more flexible basis for HRC safety than purely static safety zones, it also requires reliable perception, continuous monitoring, and suitable runtime decision-making. Therefore, existing safety measures show the importance of moving from fixed protective

2. Theoretical Background

assumptions toward context-aware risk assessment mechanisms that can consider the actual operational situation of the robot, humans, and surrounding environment.

2.1.4. Impact of Artificial Intelligence

The previous subsections show that conventional risk assessment and many existing HRC safety measures are based on pre-deployment assumptions, such as intended use, expected human behaviour, known object configurations, predefined workspaces, fixed safety zones, and foreseeable hazardous scenarios. These assumptions are necessary for systematic risk assessment, but they become less reliable when autonomous robots operate in changing environments. Human positions, object locations, obstacles, task conditions, and spatial relationships may vary during operation, meaning that a risk analysis performed before deployment may no longer fully represent the actual operating situation.

The integration of artificial intelligence into robotic perception, planning, or control further increases this limitation. AI-based components can enable object detection, scene interpretation, motion prediction, task planning, and adaptive decision-making. While these capabilities improve flexibility and autonomy, they also make robot behaviour harder to predict completely during design-time safety analysis. Safety assurance of AI-based systems is complicated by dataset dependence, black-box behaviour, explainability, and lifecycle assurance [4]. In robotic systems, this means that perception errors, low-confidence predictions, distribution shifts, previously unseen objects, changing human behaviour, or software updates after deployment can affect system decisions.

As a result, conventional risk assessment should be understood as a necessary baseline rather than a complete solution for adaptive autonomous robotic systems. If the deployed system operates under conditions that were not fully represented during the original assessment, the selected safety measures may become insufficient. Therefore, dynamic and context-aware risk assessment is required to incorporate runtime information such as robot state, human presence, object locations, spatial relationships, perception uncertainty, and environmental changes. Automated and continuous risk assessment has been proposed for software-defined robotic systems, where changes in software or system behaviour may require risk assessment to be updated during operation [6].

At the same time, AI can also support dynamic risk assessment when its outputs are structured, bounded, and interpretable. AI-based perception and reasoning methods can help identify relevant objects, detect environmental changes, classify scene elements, and localise risk-relevant information around the robot. In this thesis, this motivates the use of a spatial risk-annotated scene graph, which can represent objects, spatial relations, uncertainty, and risk-related context in a machine-readable form to support dynamic risk assessment in autonomous robotic systems.

2.2. Existing Work on Spatial Scene Graphs

Spatial scene graphs extend image-level scene graph representations by grounding objects, relationships, and higher-level spatial concepts in 3D space. This is important for robotic systems because a robot must not only recognise objects, but also understand where they are located, how they are arranged, and how they relate to larger spatial structures such as rooms or buildings. Several previous works have proposed methods for generating 3D scene graphs from reconstructed environments, RGB-D observations, or open-vocabulary perception pipelines. Although these works do not directly focus on risk annotation, they provide important methodological foundations for the spatial scene graph generation problem addressed in this thesis.

2.2.1. 3D Scene Graph by Armeni et al.

Armeni et al. introduced one of the foundational formulations of 3D scene graphs for building-scale indoor scene understanding [21]. Their work extends the conventional scene graph concept from 2D image representations to 3D environments. Instead of representing only objects and pairwise relations in an image, the proposed 3D scene graph organises semantic and spatial information across multiple levels of abstraction, including buildings, rooms, objects, and cameras.

The method uses a reconstructed 3D mesh and registered panoramic images as input. Visual information from the images is associated with the 3D mesh, allowing semantic labels and object-level information to be grounded in physical space. The resulting graph represents not only which objects are present, but also where they are located and how they are connected to larger spatial entities such as rooms. This makes the representation more suitable for indoor spatial understanding than a purely image-level graph.

A key contribution of this work is the introduction of a hierarchical representation for complete indoor environments. Object nodes can be connected to room nodes, cameras can be connected to the regions they observe, and spatial relationships can be represented explicitly. This supports queries such as which objects are present in a room, how cameras observe the environment, or how semantic information is distributed across a building.

For the present thesis, Armeni et al. are mainly relevant as a conceptual foundation for spatial scene graph structure. Their work shows how semantic information can be organised across object, room, camera, and building levels. However, the method is primarily designed for building-scale scene understanding from reconstructed data, rather than online robotic operation. Therefore, its main contribution to this thesis is the idea of spatial hierarchy, while real-time construction, loop closure, uncertainty handling, and risk annotation require additional methods.

2.2.2. SceneGraphFusion

SceneGraphFusion by Wu et al. focuses on incremental 3D scene graph prediction from RGB-D sequences [22]. This makes it particularly relevant to robotic perception, because RGB-D cameras provide both colour and depth information and are commonly used for indoor robotic systems. Unlike methods that generate a scene graph from a single image or from a completed offline reconstruction, SceneGraphFusion builds and updates the graph incrementally as new RGB-D frames are observed.

The method first uses RGB-D input to support 3D reconstruction and instance-level scene understanding. As the camera observes the environment from different viewpoints, partial observations are fused into a growing 3D representation. Object instances are represented as graph nodes, while relationships between objects are represented as graph edges. The method then predicts object and relationship semantics using graph neural network-based reasoning, combining geometric, visual, and semantic information from the reconstructed scene.

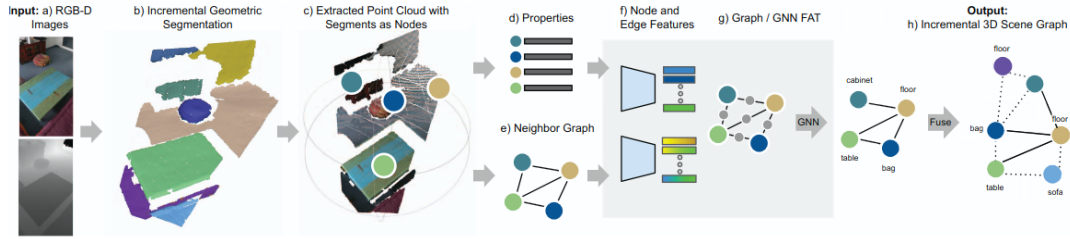


Figure 2.3.: Overview of the SceneGraphFusion pipeline, showing RGB-D input, incremental geometric segmentation, point-cloud segments as graph nodes, feature extraction, graph neural network-based relationship prediction, and temporal fusion into an incremental 3D scene graph. Reproduced from [22].

A key contribution of SceneGraphFusion is its incremental formulation. Robotic perception is naturally sequential: a robot observes only part of the environment at each time step, and the representation must be updated as new views become available. SceneGraphFusion addresses this problem by treating scene graph generation as a progressive RGB-D fusion task rather than as a single-frame prediction problem. This allows the graph to become more complete as additional observations are integrated.

An important part of the method is its temporal scene graph fusion strategy. Because the framework predicts the semantic labels of segments and relationships multiple times over a sequence, different observations may produce different or contradictory predictions. To handle this, SceneGraphFusion stores a probability distribution and a weight for each predicted class or relationship predicate. When a new prediction is available, the stored probability is updated using a weighted running average. Using $\hat{\mu}_t$ for the new prediction probability and $\hat{w}_t$ for its weight, the updated probability can be written as

2. Theoretical Background

$$\mu_t = \frac{\hat{\mu}_t \hat{w}_t + \mu_{t-1} w_{t-1}}{\hat{w}_t + w_{t-1}},$$

and the corresponding weight is updated as

$$w_t = \min(w_{\max}, \hat{w}_t + w_{t-1}),$$

where $w_{\max}$ is the maximum allowed weight [22]. This temporal fusion mechanism allows repeated observations to strengthen stable predictions while reducing the influence of isolated inconsistent predictions.

The temporal fusion strategy is relevant to this thesis as an example of how repeated observations can be combined to improve the stability of a scene graph over time. Instead of storing only a single deterministic label for each object or relationship, SceneGraphFusion preserves a probability distribution over possible classes or predicates. This allows the representation to retain information about semantic uncertainty and to reduce the effect of isolated inconsistent predictions. Such an approach is useful to study because dynamic risk assessment may depend not only on the selected object or relation label, but also on how confident the system is in that prediction.

For this thesis, SceneGraphFusion is therefore considered as an important reference for alternative approaches to incremental scene graph generation. It demonstrates how RGB-D observations can be fused over time and how semantic predictions for graph nodes and edges can be stabilized using temporal probability fusion. However, its main focus remains object-relation prediction in reconstructed indoor scenes. The present thesis investigates a different objective: extending risk-annotated scene graph generation toward spatial grounding, long-term object persistence, loop-closure-aware consistency, uncertainty representation, and dynamic risk assessment in robotic environments.

2.2.3. ConceptGraphs

ConceptGraphs by Gu et al. introduces open-vocabulary 3D scene graphs for perception and planning [8]. The work is motivated by the need for 3D scene representations that can be queried using flexible language-based concepts, rather than only a fixed set of predefined labels. This is relevant for robotic applications because the objects and categories encountered during operation may not always be known in advance. Instead of treating scene understanding only as a closed-set classification problem, ConceptGraphs connects 3D spatial mapping with language-aligned visual representations.

The method builds 3D scene graphs from posed RGB-D observations. Each RGB-D frame provides colour information, depth information, and a known camera pose. Object-level information is first extracted from individual 2D views using foundation-model-based perception. These detections

2. Theoretical Background

or segmentations provide candidate object regions in the image. The depth image and camera pose are then used to lift these 2D observations into 3D space. In this way, image-level object information is transformed into spatially grounded object candidates.

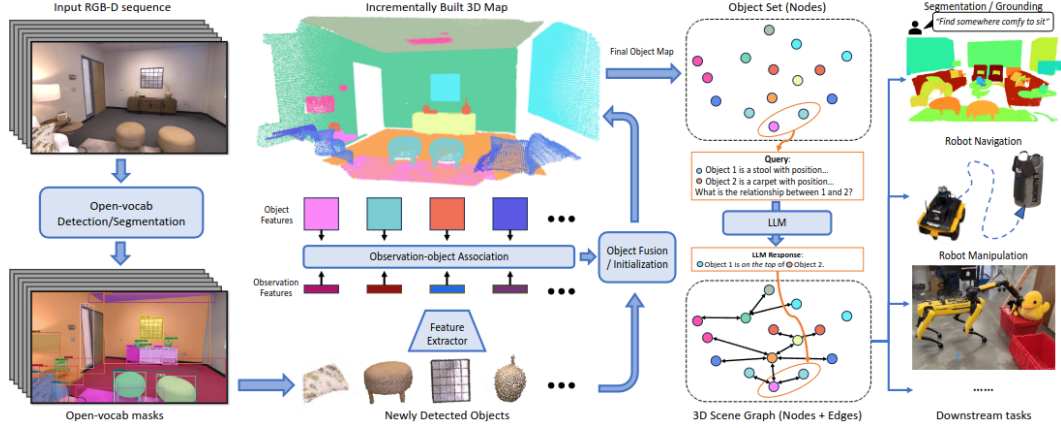


Figure 2.4.: Overview of the ConceptGraphs pipeline, showing open-vocabulary detection and segmentation from posed RGB-D observations, multi-view object association, object fusion, construction of a 3D scene graph, and downstream applications such as segmentation, grounding, navigation, and manipulation. Reproduced from [8].

A central part of the method is multi-view association. Since the same physical object can be visible in several RGB-D frames, the system must decide whether a newly detected object corresponds to an existing object in the 3D map or whether it should be inserted as a new object. ConceptGraphs performs this association using spatial and visual information, so that repeated observations of the same object can be fused into a single 3D object node. This reduces duplicate object creation and allows the graph to become more consistent as more views are processed.

The resulting graph contains object nodes that are grounded in 3D space while also carrying language-aligned features. These features allow the graph to be queried using natural-language descriptions. For example, instead of searching only for a fixed class label, a downstream module can use more flexible text queries related to object identity, appearance, or function. This makes the representation useful for tasks such as object grounding, navigation, manipulation, and language-conditioned planning.

For this thesis, ConceptGraphs is relevant because it shows one approach for combining 3D scene graph generation with foundation-model-based semantic understanding. This is related to the proposed risk-annotated scene graph framework, where visual-language reasoning is also used to enrich the scene representation. However, the focus of ConceptGraphs is mainly open-vocabulary perception and planning, whereas the present thesis focuses on generating a spatial scene graph that can support risk annotation, uncertainty representation, and dynamic risk assessment. Therefore, ConceptGraphs is useful as a reference for semantic flexibility in 3D

2. Theoretical Background

scene graphs, while the risk-oriented interpretation and safety relevance remain the focus of the proposed work.

2.2.4. OpenGraph

OpenGraph by Deng et al. proposes an open-vocabulary hierarchical 3D graph representation for large-scale outdoor environments [23]. While ConceptGraphs focuses on open-vocabulary 3D scene graphs from posed RGB-D observations, OpenGraph addresses outdoor scenes where the environment is larger, less structured, and contains many object instances. This makes the work relevant as an example of how open-vocabulary graph representations can be extended beyond indoor rooms or small-scale manipulation settings.

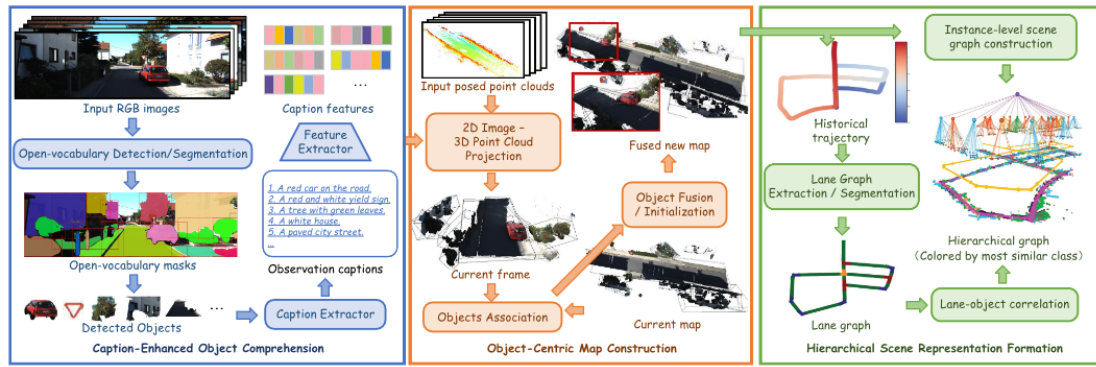


Figure 2.5.: Overview of the OpenGraph pipeline, showing caption-enhanced object comprehension from RGB images, object-centric 3D map construction using posed point clouds, and hierarchical scene representation formation for large-scale outdoor environments. Reproduced from [23].

The method first extracts object instances and textual captions from visual images. These captions are then encoded to support language-based reasoning about the detected instances. The 2D information is connected to 3D space by projecting image-based instance information onto LiDAR point clouds using multi-sensor calibration. In this way, OpenGraph builds an incremental object-centric 3D map in which objects are associated with both geometric information and language-related features.

A key feature of OpenGraph is its hierarchical organisation. The representation is not limited to isolated object nodes. Instead, the environment is structured using a hierarchy that includes object-level information and larger spatial regions derived from lane graph connectivity. This is useful in large outdoor environments, where direct object-level retrieval alone may be insufficient for navigation or spatial querying. The hierarchical graph structure allows the system to represent both local object information and broader environmental organisation.

2. Theoretical Background

For this thesis, OpenGraph is relevant because it shows how open-vocabulary semantic information can be combined with 3D graph representations in a large-scale robotic setting. Although the target environment is outdoor and the sensing setup relies on camera-LiDAR data rather than RGB-D input, the underlying idea is closely related: semantic information from visual-language models is grounded in 3D space and organised as a graph. This supports the motivation for using graph-based spatial representations when semantic reasoning must be connected to physical environments.

At the same time, OpenGraph has a different focus from the present thesis. Its main objective is open-vocabulary understanding, object retrieval, and hierarchical representation in outdoor environments. The present thesis instead focuses on spatial risk-annotated scene graph generation for robotic risk assessment. Therefore, OpenGraph is useful as a reference for large-scale and open-vocabulary graph construction, while the risk-related interpretation, uncertainty handling, and indoor RGB-D robotic integration remain separate contributions of this work.

2.3. Robot Operating System

The Robot Operating System (ROS) is an open-source software framework for developing and integrating robotic systems. Despite its name, ROS is not a traditional operating system; it functions as a middleware layer that provides communication mechanisms, software libraries, development tools, and conventions for building modular robot software [24]. In this thesis, ROS is relevant because the risk-annotated scene graph framework must be integrated with continuous RGB-D sensor streams, pose information, SLAM modules, system status information, and downstream risk assessment components. Therefore, ROS provides the conceptual and technical basis for replacing the service-oriented baseline interface with a robot-native integration approach.

2.3.1. ROS as Robotic Middleware

A ROS-based robotic system is typically organised as a collection of nodes. A node is an executable software component that performs a specific function, such as publishing camera data, processing sensor input, estimating robot pose, performing SLAM, planning motion, controlling actuators, or visualising system state. This node-based structure allows complex robotic systems to be decomposed into smaller modules that can be developed, tested, and replaced independently. For example, an RGB-D camera node can publish sensor data, a SLAM node can estimate the robot pose and map, and a scene graph node can use this information to update a structured representation of the environment.

Communication between nodes is performed using messages. A message defines the structure of the exchanged data, such as an image, pose, velocity command, point cloud, or status value. ROS provides standard message types for common robotic data, while also allowing custom message

2. Theoretical Background

definitions for application-specific information. This is important for robotic integration because different modules can exchange data in a predictable and structured way.

2.3.2. ROS Communication Mechanisms

ROS provides several communication mechanisms for different interaction patterns. Topics are used for continuous data streams and follow a publish-subscribe model. A node publishes messages to a topic, and one or more other nodes subscribe to that topic. This is suitable for sensor data such as RGB images, depth images, LiDAR scans, odometry, IMU data, and system status. In the context of this thesis, RGB-D observations, pose information, and scene graph updates can naturally be represented as topic-based communication because they are produced repeatedly during robot operation.

Services provide request-response communication. A service is useful when one node needs to request a specific operation from another node and receive a result, such as resetting a scene graph, saving the current graph, or requesting the current system state. Unlike topics, services are not intended for continuous streaming, but for discrete interactions where a response is expected.

Actions are used for long-running tasks that may require feedback or cancellation. An action consists of a goal, feedback during execution, and a final result. This mechanism is commonly used for navigation tasks, where a robot receives a goal pose, reports progress, and eventually returns whether the goal was reached. In a scene graph framework, actions could be useful for longer operations such as processing a selected dataset sequence or generating a scene graph for a defined exploration segment.

ROS also uses parameters to configure node behaviour. Parameters can define topic names, thresholds, model settings, update rates, frame identifiers, or other runtime options. This allows the same node to be reused in different experimental setups without changing the source code. Launch files are then used to start multiple nodes together with their required parameters, making it easier to reproduce complete robotic system configurations.

2.3.3. ROS 1 and ROS 2

ROS 1 was the first widely adopted version of ROS and became an important standard for robotics research and prototyping. In ROS 1, node discovery is coordinated through a central ROS Master, which allows publishers and subscribers to find each other before communicating. This architecture supported rapid development and experimentation, but it was mainly designed for research-oriented systems and did not fully address several requirements of modern robotic deployment, such as reliable distributed execution, multi-robot communication, real-time behaviour, security, and production-oriented system design.

2. Theoretical Background

ROS 2 was developed to address these limitations and to support more robust robotic applications [10]. A major architectural difference is that ROS 2 uses the Data Distribution Service (DDS) as its communication middleware. DDS enables decentralised discovery, configurable Quality of Service (QoS), and improved support for distributed robotic systems. This means that ROS 2 nodes can discover each other without relying on a central master, which improves suitability for multi-robot systems and distributed deployment.

Quality of Service is an important feature of ROS 2 communication. QoS settings allow developers to control how messages are delivered, for example whether communication should prioritize reliability or low latency. This is important because different robotic data streams have different requirements. A camera stream may prefer recent data and tolerate dropped frames, while system status or safety-related information may require more reliable delivery. ROS 2 also improves support for lifecycle management, embedded platforms, real-time control, and security compared with ROS 1 [10]. At the same time, practical studies of ROS 2 robotic control show that latency and jitter still need to be evaluated carefully in distributed and time-sensitive applications [25].

In the context of this thesis, ROS 2 is relevant because the risk-annotated scene graph framework must operate as part of a robotic system rather than as an isolated external service. RGB-D data, pose information, system status, and scene graph updates need to be exchanged continuously between different software components. ROS 2 provides a robot-native communication structure for this purpose, allowing the framework to be integrated with perception sensors, SLAM modules, robot state estimation, visualization tools, and downstream risk assessment mechanisms.

2.4. Scene Graphs

A scene graph is a structured representation of a visual scene in which objects are represented as nodes and relationships between objects are represented as edges [26]. Instead of describing an image or environment only as a collection of detected objects, a scene graph also captures how these objects are related to each other. For example, a scene may contain nodes such as *person*, *chair*, and *table*, while edges may describe relations such as *sitting on*, *next to*, or *under*. Johnson et al. demonstrated the usefulness of scene graphs for image retrieval by representing image content through objects, attributes, and relationships rather than relying only on low-level image features or isolated object labels [26].

The main advantage of a scene graph is that it provides a machine-readable representation of both entities and context. Object detections alone can indicate which objects are present, but they do not fully describe the structure of the scene. In contrast, a scene graph can represent object attributes, spatial arrangements, and semantic relationships in a compact graph format [26]. This makes it useful for tasks that require reasoning about the environment, because downstream modules can query not only whether an object exists, but also how it is positioned or related to other objects.

Scene graphs have been used in several computer vision and language-vision tasks, including image retrieval, visual relationship understanding, visual question answering, and image cap-

2. Theoretical Background

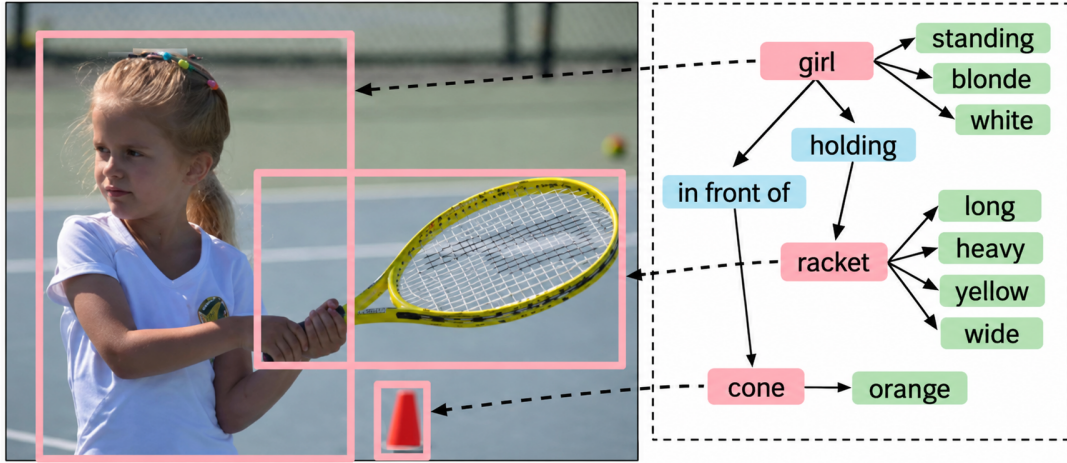


Figure 2.6.: Scene graph example. Reproduced from [26].

tioning. Datasets such as Visual Genome support this direction by providing dense annotations of objects, attributes, relationships, region descriptions, scene graphs, and question-answer pairs [27]. Beyond 2D images, scene graphs have also been extended to 3D environments, where objects, rooms, cameras, and their relationships can be represented in a unified spatial structure [21]. This makes scene graphs relevant not only for image-level understanding, but also for robotic and spatial perception tasks.

Scene graphs are also advantageous because they provide an interpretable bridge between perception and higher-level reasoning. Raw sensor data such as images or point clouds is difficult for symbolic reasoning modules to use directly. A scene graph abstracts this raw data into nodes, edges, and attributes that can be inspected, updated, and processed algorithmically. This is especially useful in robotic systems, where perception results must often be connected to planning, navigation, manipulation, or risk assessment.

In the context of this thesis, scene graphs are relevant because dynamic risk assessment requires more than object detection. Risk may depend on object identity, spatial proximity, support relations, containment, occlusion, human presence, and the current configuration of the environment. A graph-based representation allows such information to be stored in a structured form. When extended with spatial information, uncertainty estimates, and risk annotations, the scene graph becomes a suitable basis for representing the current operational context of an autonomous robotic system.

2.5. Hydra-Based Spatial Scene Graph Generation

Hydra is a real-time spatial perception system for constructing and optimizing 3D scene graphs from robotic sensor data [7]. It is relevant to this thesis because it addresses key requirements for spatial risk-annotated scene graph generation, including incremental construction, scalability to larger environments, loop closure correction, and near-real-time operation. While the baseline risk-annotated scene graph framework focuses on semantic and risk-related interpretation of RGB-D observations, Hydra provides a spatial backbone for building persistent environment-level representations.

Hydra represents the environment as a layered 3D scene graph rather than only as a set of detected objects. The graph connects metric geometry, objects, agents, places, rooms, and building-level structure. This allows the system to maintain both low-level geometric information and higher-level semantic and topological abstractions, as illustrated in Figure 2.7.

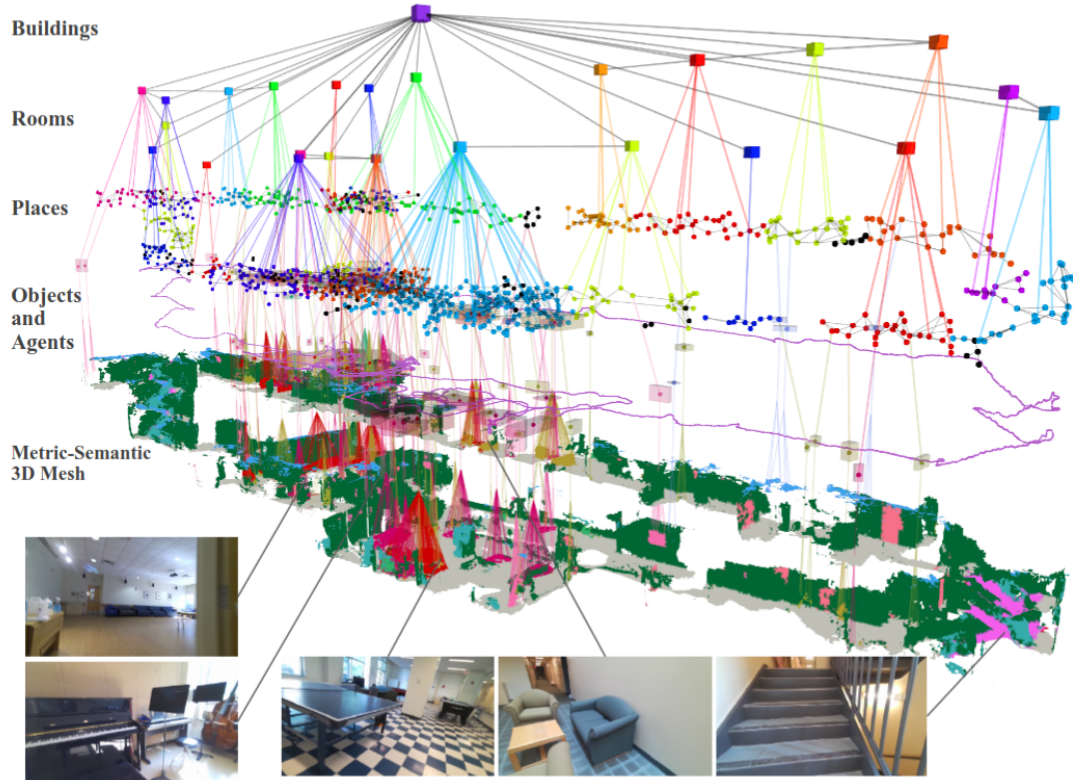


Figure 2.7.: Example of a layered 3D scene graph generated by Hydra, showing abstraction levels from metric-semantic 3D mesh to objects and agents, places, rooms, and buildings. Reproduced from [7].

2.5.1. Input Data and Processing Overview

Hydra operates on continuous sensor data from a moving robot. In the architecture described by Hughes et al., the raw inputs are images and IMU measurements, which are processed by low-level perception modules to estimate robot motion, reconstruct depth, and obtain semantic segmentation [7]. The motion estimate provides the transformations required to place observations into a common spatial reference frame, while depth information provides the 3D structure of the environment.

Semantic information is provided before the 3D scene graph layers are constructed. In simulation, these semantic labels can come from ground-truth annotations generated by the simulated environment. In real-world experiments, Hydra uses HRNet for semantic segmentation with labels from the ADE20K scene parsing dataset [7], [28], [29]. These semantic labels are combined with depth information to construct a metric-semantic 3D representation, where reconstructed geometry is associated with semantic classes such as wall, floor, furniture, or other object categories.

The overall processing pipeline transforms sensor-level data into graph-level spatial knowledge. Low-level perception estimates motion, depth, and semantic labels. Mid-level perception constructs the metric-semantic mesh, object layer, and places graph. High-level perception performs loop closure detection, scene graph optimization, and room detection. In this way, Hydra integrates observations over time to build a persistent layered representation of both the current surroundings and previously explored parts of the environment.

2.5.2. Layered 3D Scene Graph Representation

The main output of Hydra is a layered 3D scene graph, where nodes represent spatial concepts at different levels of abstraction and edges represent relationships between them. The lowest layer is the metric-semantic 3D mesh, which represents the physical geometry of the environment together with semantic labels. Above this, the object and agent layer represents detected objects and the robot trajectory. Object nodes can include semantic labels, centroids, and bounding boxes, while the agent is represented by a pose graph describing the robot’s motion [7].

The places layer provides a sparse topological representation of obstacle-free space. A place represents a traversable location in the environment, and an edge between two places indicates straight-line traversability. The room layer groups places into larger spatial regions, while the building layer provides the highest-level abstraction. This hierarchy is useful because different robotic tasks require different levels of detail: metric geometry supports local navigation, places support topological reasoning, and objects and rooms support semantic interpretation.

Hydra separates scene graph construction into a frontend and a backend. The frontend incrementally assembles an odometric scene graph from the latest mesh, object, places, and agent-pose information. The backend corrects this representation when loop closures are detected by optimizing selected graph elements and propagating the correction to the remaining layers [7].

2. Theoretical Background

This separation allows continuous local graph construction while still supporting slower global correction.

2.5.3. Incremental Construction of Spatial Layers

Hydra constructs the scene graph incrementally rather than rebuilding the full environment in a batch process. A key design choice is the use of a local active window around the robot. Within this active window, Hydra maintains volumetric spatial representations such as a Truncated Signed Distance Field (TSDF) and an Euclidean Signed Distance Function (ESDF). This limits memory usage because the system does not need to maintain an ESDF of the entire environment at all times [7].

The metric-semantic mesh is extracted from the local volumetric representation. Object nodes are then generated by clustering mesh vertices that belong to the same semantic class. For each putative object, Hydra retains the semantic label inherited from the metric-semantic mesh and estimates geometric properties such as the centroid and bounding box. New observations can be merged with existing object nodes if they overlap spatially and belong to the same semantic class; otherwise, they are inserted as new object nodes.

The places layer is extracted from the ESDF using a Generalized Voronoi Diagram (GVD). The ESDF stores the distance from each voxel, or small 3D map cell, to the nearest obstacle. Voxels that are approximately equidistant from multiple obstacles form the GVD, which lies near the medial axis of free space. In indoor environments, this means that GVD voxels tend to lie near the centre of corridors, rooms, and passages rather than close to walls or obstacles. Hydra sparsifies this structure into a graph of places by selecting representative GVD voxels as place nodes and connecting them with edges when straight-line traversal is possible. Therefore, places are not object detections; they are topological free-space locations extracted from the geometry of the environment [7].

Room detection is then performed using the places graph. The key idea is that rooms can be identified by analysing how free-space connectivity changes when obstacles are virtually inflated. When the obstacle dilation distance increases, narrow openings such as doors gradually close. This causes the places graph to split into connected components, which can correspond to individual rooms. Hydra evaluates multiple dilation distances, selects a stable partition of the places graph, and assigns remaining unlabeled places using graph clustering based on modularity. In this way, room-level structure is extracted from the topology of free space rather than from direct image-based room classification or from processing the complete dense map [7].

2.5.4. Loop Closure, Optimization, and Real-Time Architecture

As a robot moves through an environment, odometry errors can accumulate and cause drift in the estimated map and scene graph. Hydra addresses this through loop closure detection and

2. Theoretical Background

scene graph optimization. Loop closure detection uses hierarchical descriptors derived from multiple graph layers, including visual appearance, nearby object semantics, and places geometry. Candidate loop closures are then geometrically verified using visual features or object-based registration [7].

Once a loop closure is accepted, Hydra optimizes the scene graph using an embedded deformation graph. This deformation graph contains selected elements such as the agent pose graph, mesh control points, and places layer. The optimization corrects accumulated drift and propagates the correction to the rest of the scene graph. Object centroids and bounding boxes can be recomputed after deformation, overlapping nodes can be merged, and rooms can be updated from the corrected places graph.

Hydra is organized as a highly parallelized architecture with low-level, mid-level, and high-level perception processes. Low-level perception operates close to sensor rate and includes visual-inertial odometry, semantic segmentation, and depth reconstruction. Mid-level perception constructs local spatial structures such as the mesh, object layer, places graph, and scene graph frontend. High-level perception performs loop closure detection, backend optimization, and room detection. This separation prevents slower global operations from blocking faster local perception, which is important for near-real-time robotic operation.

2.5.5. Relevance to the Proposed Framework

Hydra provides a strong methodological reference for this thesis because it demonstrates how a robot can construct a persistent spatial scene graph in real time. The most relevant ideas are the layered graph representation, incremental construction of spatial layers, local active-window mapping, places extraction from free-space topology, room detection from the places graph, loop closure correction, and separation between fast local processing and slower global optimization.

In this thesis, Hydra is useful as a spatial backbone for extending the existing risk-annotated scene graph framework. The baseline framework provides semantic and risk-oriented interpretation, while Hydra provides spatial consistency, scalability, and loop closure support. Combining these ideas motivates a spatial risk-annotated scene graph that can represent risk-relevant objects, semantic relations, uncertainty, and spatial context across larger robotic environments.

2.6. Risk-Annotated Scene Graph Framework Baseline

The framework developed in this thesis extends the risk-annotated scene graph framework presented by Schiel [1], which is referred to as the baseline in the following. This section outlines its processing pipeline and external interface, explains how the scene graph is constructed and annotated with risk-relevant information, and identifies the limitations that motivate the present work. The focus lies on the conceptual design of the framework rather than on its implementation.

2. Theoretical Background

The underlying segmentation, retrieval, and vision-language techniques are introduced separately in Sections 2.7 and 2.9.

2.6.1. Information Flow in the Baseline Framework

The baseline framework follows a modular, worker-based processing pipeline in which RGB-D data is received, processed, interpreted, and integrated into a risk-annotated scene graph. The input to the system is an RGB-D observation of the environment, consisting of a colour image and its corresponding depth image, which is either provided through a dataset or submitted through the FastAPI-based interface described in Section 2.6.2. This design separates data input and client communication from the perception and scene graph generation modules. The resulting information flow is illustrated in Figure 2.8.

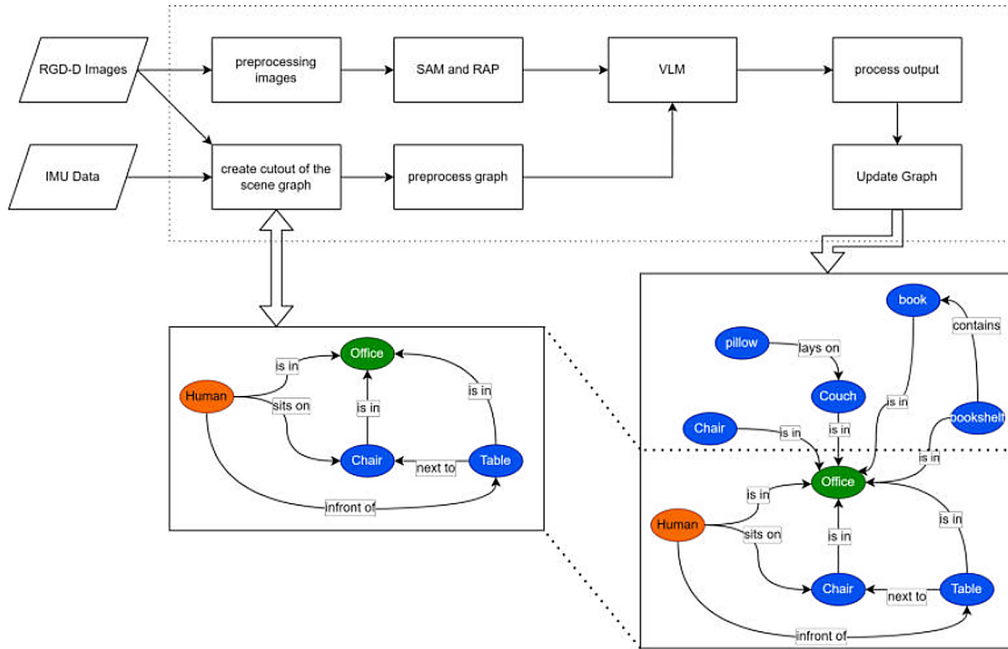


Figure 2.8.: Information flow of the existing risk-annotated scene graph framework. Adapted from [1].

Each received sample is handled as a task in the processing pipeline. The colour image is first segmented with the Segment Anything Model (SAM) [30] to obtain class-agnostic object candidates. For every retained candidate, the framework stores an image crop, a bounding box, a binary mask, and a segmentation quality score. These candidates carry no semantic meaning, so their identities must be determined in the subsequent stages. The depth image, camera parameters, and pose information are used for the later spatial processing, including coordinate conversion, visibility checking, and the spatial grounding of graph elements.

2. Theoretical Background

The object crops are then passed to the Retrieval-Augmented Perception (RAP) module, which returns label candidates from a visual knowledge base of previously stored observations (Section 2.9.1). In parallel, the existing scene graph is reduced to a cutout that contains only the nodes and edges likely to be visible in the current camera view, determined from the current pose and depth image. The RGB image, the scene-graph cutout, and the RAP label candidates are subsequently passed to a vision-language model (VLM), which returns an updated graph for the current view (Section 2.9.2). Finally, this output is grounded in 3D and merged into the complete scene graph, as described in Section 2.6.3.

2.6.2. FastAPI

FastAPI is a Python web framework for building application programming interfaces (APIs), through which external clients communicate with a software system via network-accessible endpoints, typically using HTTP requests [31]. The expected format of incoming requests can be defined explicitly, which suits complex inputs such as RGB-D images, timestamps, and pose information, and asynchronous operation allows several requests to be handled concurrently [31]. In the baseline framework, FastAPI exposes the scene graph generation pipeline as an external service and keeps client applications separated from the internal perception, segmentation, VLM, and scene graph modules. For this external communication, the internal graph is converted into a dictionary-based representation in which node and edge information are stored separately.

This interface is well suited to dataset-based testing and to request-response communication with external clients [31]. Robotic software, however, relies on continuous high-frequency sensor streams, synchronised multi-modal data, distributed nodes, and predictable timing behaviour, for which a REST-based interface is less natural. FastAPI is therefore not ideal as the primary communication mechanism for deployment on a robotic platform, which motivates the port of the framework to ROS 2 [10].

2.6.3. Scene Graph Construction and Risk Annotation

The baseline framework represents the scene graph as a directed graph, where entities in the environment are modelled as nodes and relationships between entities are modelled as edges [1]. The implementation uses NetworkX, a Python library for network analysis and graph manipulation, which supports incremental graph operations such as adding, updating, and removing nodes and edges [32].

Each node stores semantic, spatial, and risk-related information. Important node attributes include a persistent identifier, object label, semantic description, semantic pose, 3D position, risk metadata, appearance score, confidence score, and layer type. The semantic description describes the visual identity of the entity, while the semantic pose describes its current physical state, such as standing, lying, open, closed, or moving. Edges store relationships between nodes, including the source node, target node, relationship label, distance between the connected entities, and an appearance score.

2. Theoretical Background

After each VLM inference step, the returned graph is converted from image-space information into 3D coordinates using the depth image and camera pose. The converted output is then compared with the scene-graph cutout that was originally sent to the VLM, and the two are combined following an update-or-insert strategy. For each node in the VLM output, the framework checks whether a node with the same identifier already exists in the complete scene graph. If it exists, its attributes are updated using the latest VLM output and its appearance score is increased. Otherwise, it is inserted as a new node with an initial appearance score. This allows newly detected objects or humans to be added while persistent entities are reinforced over time.

Nodes that were present in the VLM input cutout but are missing from the VLM output are interpreted as not being detected in the current update. Instead of being removed immediately, their appearance score is decreased, and a node is removed only when its appearance score falls below a predefined threshold. This prevents isolated missed detections from immediately deleting objects from the graph.

The same principle is applied to edges. If an edge returned by the VLM already exists between the corresponding source and target nodes, its relationship label and distance are updated and its appearance score is increased; otherwise, a new edge is added with an initial appearance score. The edge distance is calculated from the 3D positions of the connected nodes, so that spatial relationships are stored together with semantic relationship labels. Edges that were present in the input cutout but are missing from the VLM output are weakened by decreasing their appearance score and are removed once it falls below the threshold. Temporal consistency is therefore handled through appearance-based persistence: repeatedly observed nodes and edges become more stable, while missing or transient entities are gradually removed.

2.6.4. Limitations of the Baseline Framework

When considered for deployment on a mobile robot, the baseline framework has limitations in three areas: its integration into the robotic system and the spatial extent it can represent, the reliability of the information stored in the scene graph, and its computational cost.

The first limitation arises from the service-oriented interface of the framework, which is not aligned with robot-native communication patterns (Section 2.6.2) and therefore makes deployment on real robotic platforms less straightforward. The spatial extent of the representation is likewise restricted, as the scene graph is updated based only on the current frame, camera pose, and visible graph elements. This is suitable for local scene understanding, but it provides no mechanism for keeping the graph spatially consistent across multiple rooms, corridors, or revisited locations. Since the framework also lacks loop closure, an area revisited after accumulated pose drift may lead to revisited objects being inserted as new observations, or to updates based on spatial information that is no longer globally consistent. Anchoring the scene graph in a SLAM-based spatial map with loop closure, such as the one provided by Hydra [7], is therefore necessary to extend the framework beyond local frame-by-frame scene understanding.

2. Theoretical Background

Beyond these integration aspects, the information stored in the scene graph is only partially reliable. Uncertainty is expressed solely through confidence values and appearance scores. Appearance-based persistence distinguishes repeatedly observed entities from briefly observed ones, but it maintains neither a probability distribution over object classes or relationship labels nor a probabilistic estimate of object existence or spatial position, although such information is required for downstream risk assessment (Section 2.8.4). The relationships stored on the edges are similarly limited. They are inferred by the VLM from a single two-dimensional view, and several of them, such as *in front of* or *behind*, are defined relative to the observer rather than to the scene. An object that is behind another object from one viewpoint may appear beside or in front of it from another, so the same pair of objects can receive contradictory labels as the robot moves, and each update overwrites the previously stored label. Such egocentric relations do not describe a stable property of the three-dimensional scene. Moreover, the scene graph depends directly on the VLM output and therefore inherits the failure modes described in Section 2.9.2. Since the parsing and graph-combination steps mitigate only some of these effects, the VLM output must be bounded, checked, and converted into a consistent graph representation before it can serve as reliable input for risk assessment.

Finally, the runtime performance of the framework is constrained by its per-frame inference load. Segmenting every frame with SAM in automatic mode and invoking the VLM on every frame limits the achievable update rate of the scene graph on embedded hardware, for the reasons given in Sections 2.7.1 and 2.9.2. Together, these limitations motivate the extensions proposed in this thesis: porting the framework to ROS 2, anchoring the scene graph in a SLAM-based spatial map with loop closure, introducing a probabilistic object representation, deriving object relationships from the three-dimensional geometry of the scene, constraining and validating the VLM output, and reducing the per-frame inference load to enable near-real-time operation.

2.7. Lightweight Promptable Segmentation with NanoSAM

The baseline framework uses the Segment Anything Model (SAM) to generate class-agnostic object candidates from the RGB component of each observation [1], [30]. Although SAM achieves strong zero-shot segmentation quality, its image encoder is computationally expensive. On an embedded robotic platform, segmentation must share the available compute and memory with spatial mapping, retrieval, and vision-language model inference, which makes the original model unsuitable for near-real-time operation. In this thesis, SAM is therefore replaced by NanoSAM, a distilled SAM variant developed by NVIDIA for real-time inference on Jetson devices using NVIDIA TensorRT [33], [34]. This section introduces the architecture of SAM that NanoSAM inherits, the knowledge distillation approach used to obtain NanoSAM, its deployment and performance characteristics, and its use in the proposed framework.

2.7.1. Architecture of the Segment Anything Model

SAM formulates segmentation as a promptable task: given an image and a prompt that indicates what to segment, the model returns a valid segmentation mask, even if the prompt is ambiguous [30]. The model is class-agnostic, meaning that it delineates image regions without assigning semantic labels. It consists of three components. The image encoder is a Vision Transformer (ViT) [35], pre-trained as a masked autoencoder [36], which processes an input image resized and padded to 1024×1024 pixels. The image is divided into 16×16 pixel patches, which are processed by transformer blocks using mostly windowed and occasionally global self-attention, and a convolutional neck compresses the result into a dense image embedding of $64 \times 64 \times 256$.

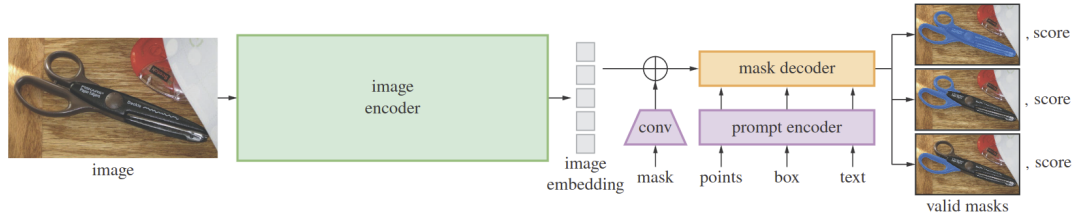


Figure 2.9.: Overview of the Segment Anything Model (SAM) architecture. The model consists of an image encoder, a prompt encoder, and a lightweight mask decoder that produces segmentation masks conditioned on input prompts. Reproduced from Kirillov et al. [30].

In this work, the model is prompted exclusively with single foreground points placed on a regular grid. The prompt encoder maps each point to a 256-dimensional token by combining a random Fourier positional encoding of its image coordinates with a learned embedding identifying it as a foreground point. Since the same positional encoding is applied to the locations of the image embedding, the prompt token and the image features share a common coordinate representation, which allows the decoder to relate the prompt to the corresponding image region. The mask decoder is a lightweight two-way transformer, in which the prompt token and a set of learned output tokens attend to each other and to the image embedding, while the image embedding in turn attends to the tokens. After upsampling the updated embedding, each output mask token is mapped to a per-pixel classifier that yields mask logits, and a dedicated output token predicts the intersection-over-union (IoU) of each mask with the target object [30].

Two design properties of SAM are particularly relevant for this thesis. First, the computational cost is distributed asymmetrically: the image encoder accounts for the large majority of the computation but is executed only once per image, after which the embedding can be reused for an arbitrary number of prompts at low additional cost. Second, since a single point can be ambiguous, for example when it lies on a part of an object as well as on the object as a whole, the decoder predicts several candidate masks at different levels of granularity together with their predicted IoU. For automatic segmentation without user input, SAM prompts the image with a regular grid of foreground points and filters the resulting masks by predicted IoU, mask stability, and non-maximum suppression (NMS). This automatic mode multiplies the decoder cost by

2. Theoretical Background

the number of grid points and, combined with the large ViT encoder, is not compatible with near-real-time operation on embedded hardware.

2.7.2. Knowledge Distillation and MobileSAM

Knowledge distillation is a model compression technique in which a compact student network is trained to reproduce the outputs or intermediate representations of a larger teacher network [37]. Since the student is supervised by the teacher rather than by ground-truth annotations, a substantial part of the teacher’s behaviour can be transferred to a model with considerably lower computational cost.

MobileSAM applies this principle to SAM [38]. Zhang et al. observed that jointly distilling the image encoder and the mask decoder is difficult, because both components are optimised in a coupled manner. They therefore proposed decoupled distillation, in which only the image encoder is distilled: a lightweight TinyViT-based encoder [39] is trained to regress the image embeddings of SAM’s ViT-H encoder. Because the student produces embeddings in the same format as the teacher, the original prompt encoder and mask decoder can be reused without retraining. In this way, MobileSAM reduces the image encoder from more than 600 million to approximately five million parameters while retaining segmentation behaviour comparable to the original model [38].

2.7.3. NanoSAM

NanoSAM extends the decoupled distillation strategy of MobileSAM by a further compression step targeted at embedded NVIDIA hardware [33]. The MobileSAM image encoder acts as the teacher, and a convolutional network acts as the student. The student uses an ImageNet-initialised ResNet-18 backbone [40], which processes the 1024×1024 input through a stem and four stages of residual blocks, progressively halving the spatial resolution and doubling the number of channels until a feature map of $32 \times 32 \times 512$ is obtained. A convolutional neck then reduces the channels, upsamples the features to 64×64 using a transposed convolution, and projects them to 256 channels. Finally, a learned positional embedding of $64 \times 64 \times 256$ is added. This positional embedding compensates for the largely translation-equivariant nature of convolutions, which would otherwise prevent the student from reproducing the position-dependent component of the transformer teacher’s embedding.

The student is trained on unlabelled images from the COCO 2017 training set [41] by regressing the teacher embedding with a Huber loss [42]. Since no segmentation annotations are required, the distillation relies entirely on the teacher’s representations. Because the embedding format is preserved, NanoSAM reuses the MobileSAM prompt encoder and mask decoder without modification. The resulting pipeline therefore consists of the distilled ResNet-18 encoder followed by the unchanged MobileSAM decoder.

2. Theoretical Background

For deployment, both components are compiled into TensorRT engines. TensorRT converts a trained network into a hardware-specific engine by applying, among other techniques, layer fusion, reduced-precision execution, and automatic selection of kernel implementations for the target GPU [34]. The NanoSAM encoder is executed in half precision (FP16), whereas the decoder is executed in single precision (FP32) [33]. Since the engines are specific to the GPU architecture and TensorRT version, they must be built on, or for, the target platform. Table 2.1 summarises the latency and accuracy reported by the developers, where accuracy is measured by prompting the model with ground-truth bounding boxes from the COCO 2017 validation set.

Table 2.1.: Latency on Jetson AGX Orin and mean IoU on COCO 2017 validation with ground-truth box prompts, as reported in [33].

Model	Latency (ms)		mIoU		
	Encoder	Full pipeline	All	Small	Large
MobileSAM	35.0	39.0	0.728	0.658	0.804
NanoSAM (ResNet-18)	4.2	8.1	0.706	0.624	0.796

2.7.4. Limitations

NanoSAM has two limitations that affect its use in the proposed framework. First, although NanoSAM uses the same decoder architecture as SAM, its TensorRT-based predictor only exposes prompt-based inference and does not include the automatic mask generation pipeline of SAM, which therefore has to be implemented separately as a post-processing wrapper around the model. In particular, object candidates must be obtained through externally generated prompts, and duplicate masks arising from neighbouring prompts must be removed by the application. Second, the distillation reduces segmentation accuracy, particularly for small objects, which may nevertheless be safety-relevant in cluttered indoor environments. Consequently, NanoSAM masks must be regarded as uncertain object proposals that require subsequent filtering, tracking, and semantic interpretation.

2.8. Multi-Object Tracking and Data Association

Segmentation, as discussed in Section 2.7, describes the objects visible in a single image, but it does not establish whether a segmented region corresponds to an object that has already been observed. For a mobile robot operating over an extended period, this distinction is essential. As the robot moves, the same physical object reappears from different viewpoints, at different distances, and

2. Theoretical Background

under varying degrees of occlusion. If every observation were treated independently, the scene representation would accumulate duplicate objects, and information gathered over time could not be attributed to the same physical entity. Multi-object tracking addresses this problem by maintaining persistent object representations over time and relating each new observation to them.

2.8.1. The Data Association Problem

The central task in multi-object tracking is data association: deciding which current observation corresponds to which previously established object. If an observation is sufficiently consistent with an existing object representation, it is associated with that object and contributes to its refinement. Otherwise, it is treated as a previously unseen instance and used to initialise a new object representation. Conversely, an existing object that is not observed in the current frame is not necessarily absent, since it may be occluded, outside the field of view, or missed by the perception system.

The reliability of this decision depends on the evidence used to compare observations with existing objects. Image-space evidence, such as the position or overlap of regions in the image, changes substantially when the camera moves, even if the observed object remains stationary. A table, for example, can occupy entirely different image regions when viewed from opposite sides of a room. For a mobile robot, apparent motion in the image is therefore caused by both object motion and ego-motion. When depth measurements and camera poses are available, observations can instead be expressed in a common three-dimensional reference frame, in which the position and extent of a static object remain approximately constant across viewpoints. Three-dimensional information therefore provides more stable evidence for associating observations that are far apart in time or taken from very different viewpoints.

Two types of association error must be considered. A false association merges observations of distinct physical objects into a single representation, while a missed association splits one physical object into duplicate representations. Both errors propagate directly into any scene representation built on top of the tracking result.

2.8.2. Object-Level SLAM with Fusion++

A relevant precedent for persistent, object-centric scene representation is Fusion++, an object-level SLAM system proposed by McCormac et al. [43]. Instead of representing the environment exclusively through a single monolithic geometric model, Fusion++ builds a persistent map composed of individual object instances. The system operates on RGB-D input and uses Mask R-CNN instance segmentation [44] to detect object regions in incoming frames. The corresponding depth measurements are used to initialise a separate volumetric TSDF model for each newly

2. Theoretical Background

detected object, with the voxel resolution of each model chosen according to the object’s size. A coarse TSDF of the background is maintained in parallel to support camera tracking [43].

Data association in Fusion++ couples the persistent 3D models with the 2D segmentation output. Existing object models are rendered into the current camera view, and each new instance mask is compared with these renderings using their intersection-over-union. A mask is associated with an existing object if the overlap exceeds a threshold. Otherwise, it initialises a new object [43]. When an object is observed again, the new depth measurements are fused into its existing TSDF rather than creating an independent copy. Since an object is rarely fully visible from a single viewpoint, repeated observations from different viewpoints gradually complete its geometry. Persistent association therefore serves two purposes: it preserves object identity, and it allows the spatial representation of each object to be refined over time.

Fusion++ also maintains uncertainty-related quantities for each object. Semantic identity is not fixed at initialisation. Instead, the class probabilities predicted by Mask R-CNN are fused over successive observations, so that the class estimate of an object can change as further evidence becomes available, independently of the object’s identity in the map. In addition, each object carries an existence probability, which is derived from how often the object is detected compared with how often it is expected to be visible but is not detected. Objects whose existence probability falls too low are removed, which suppresses spurious detections without discarding objects after a single missed observation [43]. Finally, the object instances are organised in a six-degree-of-freedom pose graph and used as landmarks for tracking, relocalisation, and loop-closure detection. As a result, objects become persistent elements of the spatial map rather than transient detections tied to individual frames.

2.8.3. Limitations of Fusion++

Despite these properties, Fusion++ is not suitable as the basis of the proposed framework. Its semantic perception is closed-set, since objects are detected and classified by Mask R-CNN, which is trained on a fixed set of categories [43]. Objects outside this set are not represented as distinct entities, which conflicts with the requirement of open-vocabulary identification of risk-relevant objects. Furthermore, Fusion++ produces an object-level map rather than a scene graph. It contains no relationships between objects, descriptive object properties, or higher-level spatial structures such as rooms, and therefore lacks the relational context on which downstream safety reasoning depends.

In addition, Fusion++ assumes a static environment [43]. Its existence probability indicates whether an object is repeatedly detected, but not whether a previously observed object is still at its last observed position, even though this uncertainty is itself risk-relevant in dynamic environments. Finally, maintaining a separate volumetric model for every object, combined with a two-stage segmentation network, results in memory and computational costs that grow with the

2. Theoretical Background

number of objects. This conflicts with the requirement of near-real-time operation on embedded hardware.

2.8.4. Relevance for Persistent Scene Representations

Despite these limitations, the central idea behind Fusion++ remains relevant to this work. Image observations can be combined with depth to build persistent object representations, and new observations can be linked back to them over time. For a scene graph that supports risk assessment, this means that each node should represent a physical object rather than a single detection. Information collected at different times, such as labels, properties, or positions, can then be gathered in one place instead of being spread across duplicate entries. The semantic label of an object should also be treated as an estimate that can improve with more observations, rather than as a fixed value assigned when the object is first seen.

In the same way, the presence of an object is better described by a probability than by a simple yes or no. This makes it possible to tell apart objects that have been seen many times, objects that were seen only briefly, and objects that have not been observed for some time. Such distinctions matter for risk assessment, since an object that may have moved should be treated differently from one whose position is well known. These ideas, however, need to be applied under conditions that Fusion++ does not consider, including open-vocabulary object identification, relations between objects, changing object positions, and near-real-time operation on embedded hardware.

2.9. Visual Retrieval and Vision-Language Models

Class-agnostic segmentation produces object masks without semantic meaning. Labels can be assigned to such masks by visual retrieval, which reuses the label of a previously seen, similar object, or by a vision-language model (VLM), which names an object directly from its appearance. Retrieval is computationally cheap but limited to what has been observed before, whereas a VLM can describe arbitrary objects at a considerably higher computational cost. The baseline framework combines both approaches [1], and its concept forms the starting point for the pipeline developed in this thesis.

2.9.1. Retrieval-Augmented Perception

Visual retrieval requires an image encoder that converts an image into a fixed-length vector, referred to as an embedding, such that similar images obtain similar embeddings. A widely used encoder is the image encoder of Contrastive Language–Image Pre-training (CLIP) [45]. In its ViT-B/32 variant, the image is resized to 224×224 pixels, divided into 32×32 pixel patches, and

2. Theoretical Background

processed by a Vision Transformer [35]. A summary token that collects information from all patches is projected to a vector of 512 values and normalised to unit length.

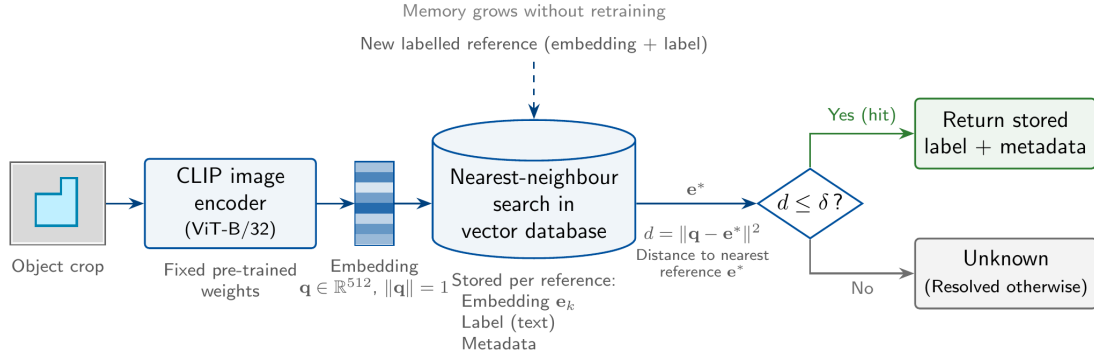


Figure 2.10.: Principle of visual retrieval: the crop embedding is compared with stored references, and the nearest label is returned if its distance lies below the threshold δ .

The meaning of these embeddings results from the encoder weights, which were determined once during pre-training and remain fixed afterwards. CLIP was trained on approximately 400 million images with their captions; during training only, a separate text encoder maps each caption into the same space, and both networks are optimised so that each image lies close to its own caption and far from all others [45]. To achieve this, the image encoder must extract features that describe objects, materials, and scenes. Images with similar content must lie close to similar captions and are therefore also close to each other. After training, images are compared using the image encoder alone. To compare a new image with a stored reference image, their embeddings are compared. Let $\mathbf{q}$ denote the embedding of the new image (the query) and $\mathbf{e}$ the embedding of a stored reference. Their squared Euclidean distance $\|\mathbf{q} - \mathbf{e}\|^2$, i.e. the sum of the squared differences of all 512 values, is directly related to their cosine similarity $\mathbf{q}^\top \mathbf{e}$. Since both embeddings have unit length,

$$\|\mathbf{q} - \mathbf{e}\|^2 = \|\mathbf{q}\|^2 + \|\mathbf{e}\|^2 - 2\mathbf{q}^\top \mathbf{e} = 2(1 - \mathbf{q}^\top \mathbf{e}). \quad (2.1)$$

The distance is therefore zero for identical embeddings and increases as the cosine similarity decreases, so both measures rank stored references in the same order.

Retrieval-augmented methods use stored knowledge to support the interpretation of new input [46]. In visual retrieval, a memory stores the embedding of each labelled reference image together with its label as plain text and optional metadata. Vector databases such as ChromaDB organise these embeddings in an approximate nearest-neighbour index for efficient search [47]. A new image is embedded with the same encoder, and if the distance to the nearest reference lies below a threshold, the stored label is returned; otherwise, the image remains unknown. The memory thus does not interpret images itself but determines whether an image closely resembles one that has already been labelled. New objects can be added at any time without retraining, since the learning takes place in the stored references rather than in the encoder.

2. Theoretical Background

In the baseline framework, this principle is implemented as Retrieval-Augmented Perception (RAP) [1]. For each frame, the object cutouts generated by the Segment Anything Model are embedded with CLIP and compared with a visual knowledge base in ChromaDB. The labels of sufficiently similar references are passed to the VLM as label candidates, which guide its interpretation and help to keep labels consistent across observations. RAP therefore supports the VLM in every inference step but does not replace it. The same principle could also be used to resolve an object directly when a sufficiently similar reference exists, so that the VLM would only be required for the remaining objects.

Retrieval is fast, and repeated observations of the same object receive consistent labels. However, it can only return labels that are already stored, an incorrect entry is propagated to all similar observations, and the threshold trades the number of resolved objects against the number of accepted errors. Moreover, the encoder summarises the entire image including its background, because nothing indicates which pixels belong to the object. Large uniform surfaces such as walls and ceilings in the same room can therefore appear more similar to each other than to the same surface type elsewhere.

2.9.2. Vision-Language Models

A VLM consists of a vision encoder, which converts image patches into feature vectors, a connector, which maps these features into the input space of a large language model, and the language model, which generates the answer token by token from the visual input and a text prompt [48]. Since each token requires one forward pass, the latency grows with the length of the answer. A pre-trained VLM can be applied to new tasks without retraining, as the task is specified in the prompt.

Compared with conventional object detectors and segmentation networks, which are trained on a fixed set of classes, VLMs are open-vocabulary: they can name objects, materials, and states that were never defined as classes in advance, and they can describe properties beyond the object category, such as the condition of an object or its relation to its surroundings. This capability results from pre-training on large and diverse image-text data. At the same time, the generated text is not restricted to what is visible in the image. A well-documented failure mode is object hallucination, in which a model describes objects or attributes that are not present, often because they frequently co-occur with the visible content in the training data [49]. The flexibility of VLMs is therefore accompanied by the risk of plausible but incorrect answers.

In the baseline framework, the VLM is the central reasoning component [1]. For each frame, it receives the RGB image, the part of the existing scene graph expected to be visible, and the label candidates from RAP, and returns an updated scene graph with labels, descriptions, states, confidence values, risk-relevant properties, and relationships. The output is parsed, converted into 3D coordinates, and merged into the scene graph. The framework is model-interchangeable, and the study reports formatting errors, repetitions, hallucinated relationships, and unstable location output as typical difficulties [1]. Since the VLM is called for every frame, its inference time also limits the update rate of the scene graph.

2. Theoretical Background

In general, VLM output cannot be treated as ground truth. A reported confidence value is generated text rather than a calibrated probability, and models often state high confidence for incorrect answers [50]. VLM output should therefore be validated, for example against confidence thresholds, before it is stored in a persistent environment representation.

2.9.3. The Qwen Vision-Language Model Family

The baseline study evaluated three VLMs for scene graph generation: LLaVA 1.6 Mistral 7B, Qwen2.5-VL 7B, and Phi-4 Multimodal [1]. The models were compared with respect to the compatibility of their input and output formats, their tendency towards repetition and hallucination, and the stability of the generated object locations. Based on this comparison, the Qwen series was identified as the most suitable model family, and later tests of the baseline framework used the newer Qwen3-VL generation, taking advantage of the model-interchangeable design of the framework [1]. This thesis builds on this result. Instead of repeating the comparison between model families, it continues with the best-performing Qwen family and updates the model choice to the latest available generations, with a focus on small models that are suited to embedded hardware.

The Qwen models of Alibaba Cloud are published with open weights in several sizes and can therefore be executed locally on embedded hardware. Table 2.2 lists the relevant generations.

Table 2.2.: Release history of the open-weight Qwen vision-language models.

Generation	Release	Small dense sizes
Qwen2-VL	Aug. 2024	2B, 7B
Qwen2.5-VL	Jan. 2025	3B, 7B
Qwen3-VL	Oct. 2025	2B, 4B, 8B
Qwen3.5	Mar. 2026	0.8B, 2B, 4B, 9B

Qwen2-VL introduced image processing at native resolution [51], and Qwen2.5-VL improved efficiency, object localisation, and document understanding [52]. Qwen3-VL builds on the Qwen3 language models and supports contexts of up to 256K tokens [53]. Qwen3.5 is trained natively on mixed image and text data instead of attaching a vision encoder to a separately trained language model, and by default generates a reasoning trace before its answer, which can improve accuracy but increases latency [54].

The released models differ in size, architecture, and operating mode. Dense models use all of their parameters for every token, whereas mixture-of-experts models activate only a subset of their parameters per token, which reduces computation but not the memory required to store the full model [53]. In addition, Qwen3-VL is released in an instruction-following variant, which answers directly, and a thinking variant, which reasons explicitly before answering [53], while Qwen3.5 combines both behaviours in one model [54]. For embedded deployment, the memory footprint is decisive. The memory required for the weights is approximately the number of parameters

2. Theoretical Background

multiplied by the number of bits per parameter, so that an 8B model in 16-bit precision requires about 16 GB, whereas a 4B model stored with about 4 bits per parameter requires roughly 2 to 3 GB. Small dense models, which are commonly also distributed in such reduced-precision formats, are therefore the most suitable candidates when the VLM has to run alongside other perception and mapping components on the same device.

Since the baseline study, a new generation has appeared roughly every six to twelve months, and small models have improved noticeably: on the MMMU-Pro benchmark, Qwen3.5-4B scores 65.4 % compared with 52.0 % for Qwen3-VL-4B [55]. Although general benchmarks do not directly predict task-specific accuracy, this trend indicates that the small Qwen3-VL and Qwen3.5 models are promising candidates for the present work, and that future generations may improve the results further. The model-interchangeable design of the baseline framework allows such newer models to be adopted without changes to the remaining pipeline.

2.10. Embedded Computing Hardware: NVIDIA Jetson Platforms

The proposed framework combines several computationally demanding components, including NanoSAM-based segmentation, retrieval-augmented perception, VLM reasoning, Hydra-based spatial mapping, and risk-annotated scene graph generation. For onboard robotic deployment, these components must run on embedded hardware with sufficient AI compute, memory capacity, and energy efficiency. This section introduces the two NVIDIA Jetson platforms relevant to this work, the Jetson AGX Orin Developer Kit and the Jetson AGX Thor Developer Kit.

2.10.1. NVIDIA Jetson AGX Orin Developer Kit

NVIDIA Jetson AGX Orin is an embedded edge-AI computing platform designed for robotics, computer vision, and autonomous machines. Like other members of the Jetson family, it integrates CPU, GPU, memory, I/O, and AI acceleration into a compact module for onboard computation. Jetson AGX Orin is based on the NVIDIA Ampere architecture, combining a 2048-core GPU with 64 Tensor Cores and a 12-core Arm Cortex-A78AE CPU. It provides up to 275 TOPS of AI performance, 64 GB of memory, and a configurable power range of 15 W to 60 W [56].

Jetson AGX Orin was used as the target platform in the baseline framework [57]. Its compute capacity is sufficient for executing individual perception and reasoning components, such as segmentation and VLM inference with models in the 7B parameter class. However, executing the complete pipeline, consisting of segmentation, retrieval, VLM reasoning, and spatial mapping, concurrently on a single module can be expected to place considerable demands on the available memory and compute resources. Since the CPU and GPU share the same physical memory and power budget, these components compete for the same resources, which may limit the achievable update rate of the scene graph.

2.10.2. NVIDIA Jetson AGX Thor Developer Kit

NVIDIA Jetson AGX Thor is the successor platform within the Jetson family and is designed specifically for physical AI, robotics, and real-time multimodal perception. It is based on the NVIDIA Blackwell architecture, combining a 2560-core GPU with 96 fifth-generation Tensor Cores and a 14-core Arm Neoverse-V3AE CPU. It provides up to 2070 FP4 sparse TFLOPS of AI compute, 128 GB of memory, and a configurable power range of 40 W to 130 W [58]. According to the manufacturer, Jetson AGX Thor provides up to 7.5 times higher AI compute and 3.5 times better energy efficiency than Jetson AGX Orin [58].¹

This additional capacity is potentially relevant for the proposed framework, as it may allow the perception, reasoning, and mapping components to be executed concurrently on the same embedded platform with fewer resource constraints. Jetson AGX Thor also offers improved support for transformer-based and multimodal workloads. Jetson AI Lab lists Qwen3-VL models, including Qwen3-VL 4B and Qwen3-VL 8B, as available for Jetson AGX Thor through Quick Start Runner and vLLM-based serving configurations [59]. Since the Qwen model series was identified as the best-performing option in the baseline framework [57], Jetson AGX Thor represents a suitable hardware basis for extending the system toward onboard robotic deployment without dependence on an external workstation for model inference.

2.11. Scene Graphs for Downstream Safety Reasoning

This section discusses how a generated scene graph can be used as structured input for downstream risk assessment. SafetyDetect by Mullen et al. is considered as an example of scene-graph-based safety reasoning [9]. The work proposes a dataset and LLM-based approach for detecting anomalous situations in home environments, such as food left outside storage, a stove left on, or harmful objects being accessible to children. In the proposed pipeline, object relations from a scene graph are provided to an LLM, which classifies situations as normal, dangerous, unsanitary, or dangerous for children.

The relevance of SafetyDetect lies in its demonstration that object relations can be important for safety-related reasoning. Many risks cannot be inferred from object identity alone, because the same object may be safe or unsafe depending on its relation to other objects, humans, or locations. A scene graph is therefore useful because it represents these relations explicitly and provides a structured intermediate representation for a downstream reasoning module.

At the same time, SafetyDetect highlights the importance of the scene graph generation stage. The authors state that their method is limited by perception, particularly by how the scene graph is created [9]. For example, a scene graph may represent that an object is inside a room and on the floor, but it may not indicate whether the object is blocking a doorway and creating a

¹The compute figures of the two platforms are specified at different numerical precisions (FP4 sparse TFLOPS for Jetson AGX Thor and TOPS for Jetson AGX Orin). The comparison therefore reflects the manufacturer’s specification rather than a measured result.

2. Theoretical Background

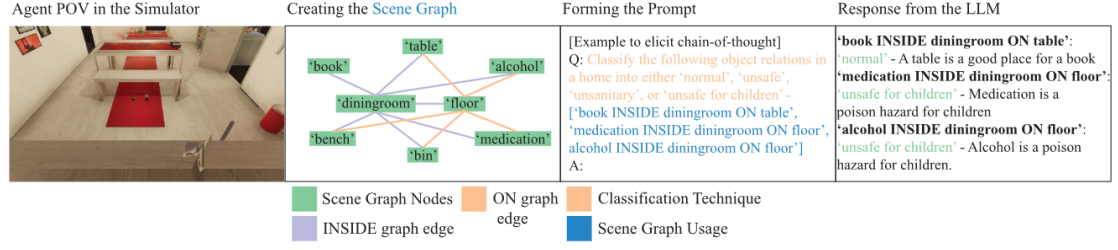


Figure 2.11.: SafetyDetect pipeline showing the agent point of view, scene graph construction, prompt formation, and LLM-based anomaly classification. Reproduced from [9].

tripping hazard. This shows that the downstream safety assessment depends not only on the reasoning capability of the LLM, but also on whether the scene graph contains the required objects, relations, spatial details, and obstruction information.

This limitation is important for the present thesis. SafetyDetect mainly focuses on reasoning after a scene graph has already been generated. In simulation, the scene graph is provided by the VirtualHome environment, while in the real-world TurtleBot experiment an existing scene graph generation method is used before the graph is passed to the LLM. Therefore, SafetyDetect is more relevant as an example of downstream safety reasoning than as a method for constructing a persistent spatial scene graph.

Consequently, SafetyDetect can be understood as evidence for the downstream value of scene-graph-based representations in safety reasoning. However, it also indicates that the effectiveness of such reasoning depends strongly on the quality of the generated scene graph. This supports the central motivation of the present thesis: improving scene graph generation, spatial grounding, persistence, and uncertainty representation is a necessary step toward more reliable dynamic risk assessment in robotic systems.

3. Methodology

3.1. Baseline System and Proposed Extensions

Hydra builds the 3D map and publishes local graph context in real time. The VLM pipeline asynchronously consumes this context together with RGB-D frames, produces object/risk observations, and publishes them back. A Hydra-side fusion module validates and integrates these observations into the global Dynamic Scene Graph.

3.2. Hardware Setup and Input Data Preparation

This section describes the hardware setup used for data collection and the preparation of input data for the proposed risk-annotated scene graph pipeline. In a complete deployment scenario, the system is expected to operate on a mobile robot equipped with an RGB-D camera and sufficient onboard computing resources to run the full perception, mapping, and risk-assessment pipeline in real time. Such a setup would allow sensor data acquisition, semantic interpretation, scene graph construction, and risk annotation to be performed directly on the robot.

For the current development and evaluation phase, however, the available hardware is distributed across three separate components: the RGB-D sensing unit, the mobile robotic platform, and the external processing unit. The RGB-D sensor used in this work is an Intel RealSense D455 camera, which provides synchronized color and depth information. The mobile platform is a Unitree Go1 quadruped robot, which provides the physical mobility required for collecting data in indoor environments. The processing unit is an NVIDIA Jetson Thor, which is intended to execute the computationally demanding components of the pipeline, such as visual-language-model-based reasoning and semantic scene graph processing.

In the present setup, the Unitree Go1 does not provide the required hardware interface to power and operate the NVIDIA Jetson Thor directly as an onboard edge-computing

3. Methodology

device. Therefore, the system is used primarily for data collection, while the computationally demanding components of the pipeline are executed later on the NVIDIA Jetson Thor. The Intel RealSense D455 is rigidly mounted on the Unitree Go1 using a 3D-printed frame and connected to the robot’s onboard computer through USB. During operation, the RGB-D sensor data, camera calibration information, and robot odometry are transmitted to a local computer through Ethernet and recorded as a ROS bag. If required, additional camera sensor information, such as IMU measurements from the RealSense device, can also be included in the recorded dataset for future pose-estimation or sensor-fusion extensions. The recorded data can then be replayed at real-time speed for offline processing, testing, and debugging. This workflow is useful during development because the perception and risk-assessment pipeline can be evaluated repeatedly without physically operating the robot for every debugging cycle.

Although wireless transmission of the sensor data is technically possible, it was not selected as the primary data-transfer method because RGB-D streams require high and stable bandwidth. In particular, transmitting raw color images, aligned depth images, and odometry data over Wi-Fi can lead to frame drops, variable latency, and reduced synchronization reliability. Since accurate temporal alignment between RGB-D frames and robot odometry is important for later preprocessing and 3D scene reconstruction, Ethernet is preferred for data collection. This provides a more stable connection between the robot and the local recording computer and reduces the risk of incomplete or inconsistent ROS bag recordings.

From a software perspective, the hardware setup also requires an interface between two ROS versions. The Unitree Go1 currently operates using a ROS 1-based software stack, while the proposed risk-annotated scene graph pipeline is developed in ROS 2. Therefore, during data collection, both the RealSense D455 driver and the Unitree Go1 odometry interface are operated within the ROS 1 environment on the robot side. The RGB-D data, camera calibration information, and robot odometry are transmitted to the local computer and stored as a ROS 1 bag. For offline processing, this recorded data is converted into a ROS 2-compatible bag format before being replayed through the proposed ROS 2 pipeline. This also implies that, for an actual deployment of the complete system, the parent robotic platform should either provide a native ROS 2 software interface or use a dedicated ROS 1-to-ROS 2 bridge to transfer the required sensor and odometry data into the ROS 2 processing pipeline in real time.

3.3. Data Preprocessing

After the recorded data has been converted into a ROS 2-compatible format, it is replayed as timestamped sensor and odometry streams. However, these individual streams cannot be used directly by the risk-annotated scene graph pipeline, since the downstream modules require each input sample to contain a consistent RGB-D-pose representation. The purpose of the preprocessing stage is therefore to convert the available sensor and odometry data into a structured frame representation that contains the visual observation, depth information, camera calibration, and camera pose required for later semantic and spatial reasoning.

The first requirement of preprocessing is temporal consistency. The RGB image, aligned depth image, camera calibration, and odometry data must correspond to the same observation time. If these data sources are not correctly associated, the resulting 3D interpretation may become inaccurate, since image regions would be projected using depth or pose information from a different time instant. Therefore, only frames for which the required sensor and pose information can be reliably associated are forwarded to the subsequent processing stages.

The second requirement is format consistency. The RGB image is converted into a uniform color representation, while the depth image is converted into metric units. Since later stages use image regions from the RGB image together with corresponding depth values, aligned depth data are used so that RGB and depth pixels refer to the same image coordinates. Invalid or out-of-range depth values are also identified during preprocessing, since unreliable depth information can negatively affect object localization and scene graph construction.

The camera calibration information is preserved together with each processed frame. This information is required to project image-based observations into 3D space. In addition, each frame is associated with a camera pose derived from the robot odometry and the fixed relationship between the robot body and the mounted camera. This provides the translation and rotation information required by the scene graph pipeline for spatial reasoning. Optional inertial measurements from the camera can also be preserved for future extensions, such as visual-inertial odometry or pose fusion. However, in the current preprocessing formulation, the camera is assumed to be rigidly mounted, and the camera pose is derived from robot odometry together with the fixed mounting relationship.

The output of preprocessing is a prepared RGB-D-pose frame containing the synchronized image, depth, calibration, pose, timestamp, and frame identifier. This representation forms the input to the main processing stage, where segmentation, visual retrieval,

3. Methodology

visual-language-model inference, scene graph updates, and risk annotation are performed. By separating data preparation from semantic reasoning, the system maintains a clear boundary between sensor-data handling and higher-level interpretation.

3.4. Pre-Processing

4. Experiments

5. Conclusions

Bibliography

- [1] André Schiel, “Online Risk-Annotated Scene Graphs for Autonomous Systems,” Master’s thesis, University of Stuttgart, Stuttgart, Germany, 2026.
- [2] International Federation of Robotics. “Service robots see global growth boom.” Press release on World Robotics 2025 – Service Robots. (Oct. 7, 2025), [Online]. Available: <https://ifr.org/ifr-press-releases/news/service-robots-see-global-growth-boom> (visited on 09/28/2026).
- [3] International Federation of Robotics, *World robotics 2025: Press conference presentation*, 2025. [Online]. Available: https://ifr.org/downloads/press_docs/PressConference2025_presentation.pdf.
- [4] A. V. Silva Neto, J. B. Camargo, J. R. Almeida, and P. S. Cugnasca, “Safety assurance of artificial intelligence-based systems: A systematic literature review on the state of the art and guidelines for future work,” *IEEE Access*, vol. 10, pp. 130 733–130 770, 2022. doi: 10.1109/ACCESS.2022.3229233.
- [5] International Organization for Standardization, *Iso 12100: Safety of machinery – general principles for design – risk assessment and risk reduction*, International Organization for Standardization, 2010.
- [6] P. Grimmeisen, R. Golwalkar, Y. Ma, and A. Morozov, “Automated and continuous risk assessment for ros-based software-defined robotic systems,” in *2023 IEEE 19th International Conference on Automation Science and Engineering (CASE)*, IEEE, 2023, pp. 1–7. doi: 10.1109/CASE56687.2023.10260416.
- [7] N. Hughes, Y. Chang, and L. Carlone, “Hydra: A real-time spatial perception system for 3d scene graph construction and optimization,” in *Robotics: Science and Systems*, New York City, NY, USA, 2022.
- [8] Q. Gu, A. Kuwajerwala, S. Morin, *et al.*, “Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning,” in *2024 IEEE International Conference on Robotics and Automation*, IEEE, 2024, pp. 5021–5028. doi: 10.1109/ICRA57147.2024.10610243.
- [9] J. F. J. Mullen, P. Goyal, R. Piramuthu, M. Johnston, D. Manocha, and R. Ghanadan, ““don’t forget to put the milk back!” dataset for enabling embodied agents to detect anomalous situations,” *IEEE Robotics and Automation Letters*, vol. 9, no. 10, pp. 9087–9094, 2024. doi: 10.1109/LRA.2024.3430129.

Bibliography

- [10] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, eabm6074, 2022. doi: 10.1126/scirobotics.abm6074.
- [11] A. Rosinol, A. Violette, M. Abate, N. Hughes, Y. Chang, J. Shi, A. Gupta, and L. Carlone, "Kimera: From slam to spatial perception with 3d dynamic scene graphs," *The International Journal of Robotics Research*, vol. 40, no. 12–14, pp. 1510–1546, 2021. doi: 10.1177/02783649211056674.
- [12] X. Shi, D. Li, P. Zhao, Q. Tian, Y. Tian, Q. Long, C. Zhu, J. Song, F. Qiao, L. Song, *et al.*, "Are we ready for service robots? The OpenLORIS-Scene datasets for lifelong SLAM," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2020, pp. 3139–3145. doi: 10.1109/ICRA40945.2020.9196638.
- [13] M. Osborne, R. D. Hawkins, and J. A. McDermid, "Analysing the safety of decision-making in autonomous systems," in *Computer Safety, Reliability, and Security*, ser. Lecture Notes in Computer Science, vol. 13414, Springer, 2022, pp. 3–16. doi: 10.1007/978-3-031-14835-4_1.
- [14] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," *IEEE Transactions on Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, 2004. doi: 10.1109/TDSC.2004.2.
- [15] International Organization for Standardization, *Iso 10218-1:2025 robotics – safety requirements – part 1: Industrial robots*, International Organization for Standardization, 2025. [Online]. Available: <https://www.iso.org/standard/73933.html> (visited on 05/25/2026).
- [16] International Organization for Standardization, *Iso 10218-2:2025 robotics – safety requirements – part 2: Industrial robot applications and robot cells*, International Organization for Standardization, 2025. [Online]. Available: <https://www.iso.org/standard/73934.html> (visited on 05/25/2026).
- [17] International Organization for Standardization, *Iso/ts 15066:2016 robots and robotic devices – collaborative robots*, International Organization for Standardization, 2016. [Online]. Available: <https://www.iso.org/standard/62996.html> (visited on 05/25/2026).
- [18] European Commission, "Industry 5.0: Towards a sustainable, human-centric and resilient european industry," European Commission, 2021. [Online]. Available: <https://op.europa.eu/en/publication-detail/-/publication/468a892a-5097-11eb-b59f-01aa75ed71a1> (visited on 05/25/2026).
- [19] S. P. Kumar, C. Savur, and F. Sahin, "Survey of human–robot collaboration in industrial settings: Awareness, intelligence, and compliance," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 51, no. 1, pp. 280–297, 2021. doi: 10.1109/TSMC.2020.3041231.

Bibliography

- [20] J. A. Marvel, "Performance metrics of speed and separation monitoring in shared workspaces," *IEEE Transactions on Automation Science and Engineering*, vol. 10, no. 2, pp. 405–414, 2013. doi: 10.1109/TASE.2013.2237904.
- [21] I. Armeni, Z.-Y. He, J. Gwak, A. R. Zamir, M. Fischer, J. Malik, and S. Savarese, "3d scene graph: A structure for unified semantics, 3d space, and camera," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 5664–5673.
- [22] S.-C. Wu, J. Wald, K. Tateno, N. Navab, and F. Tombari, "Scenegraphfusion: Incremental 3d scene graph prediction from rgb-d sequences," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 7515–7525.
- [23] Y. Deng, J. Wang, J. Zhao, X. Tian, G. Chen, Y. Yang, and Y. Yue, "Opengraph: Open-vocabulary hierarchical 3d graph representation in large-scale outdoor environments," *IEEE Robotics and Automation Letters*, vol. 9, no. 10, pp. 8402–8409, 2024. doi: 10.1109/LRA.2024.3439122.
- [24] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "Ros: An open-source robot operating system," in *ICRA Workshop on Open Source Software*, 2009.
- [25] L. Puck, P. Keller, T. Schnell, C. Plasberg, A. Tanev, G. Heppner, A. Roennau, and R. Dillmann, "Performance evaluation of real-time ros2 robotic control in a time-synchronized distributed network," in *2021 IEEE 17th International Conference on Automation Science and Engineering (CASE)*, IEEE, 2021, pp. 1670–1676. doi: 10.1109/CASE49439.2021.9551447.
- [26] J. Johnson, R. Krishna, M. Stark, L.-J. Li, D. A. Shamma, M. S. Bernstein, and L. Fei-Fei, "Image retrieval using scene graphs," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 3668–3678. doi: 10.1109/CVPR.2015.7298990.
- [27] R. Krishna, Y. Zhu, O. Groth, *et al.*, "Visual genome: Connecting language and vision using crowdsourced dense image annotations," *International Journal of Computer Vision*, vol. 123, pp. 32–73, 2017. doi: 10.1007/s11263-016-0981-7.
- [28] J. Wang, K. Sun, T. Cheng, *et al.*, "Deep high-resolution representation learning for visual recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 10, pp. 3349–3364, 2021. doi: 10.1109/TPAMI.2020.2983686.
- [29] B. Zhou, H. Zhao, X. Puig, S. Fidler, A. Barriuso, and A. Torralba, "Scene parsing through ade20k dataset," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 633–641. doi: 10.1109/CVPR.2017.544.
- [30] A. Kirillov, E. Mintun, N. Ravi, *et al.*, "Segment anything," *arXiv preprint arXiv:2304.02643*, 2023. arXiv: 2304.02643 [cs.CV].

Bibliography

- [31] FastAPI. “Fastapi documentation.” (2026), [Online]. Available: <https://fastapi.tiangolo.com/> (visited on 05/28/2026).
- [32] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using networkx,” in *Proceedings of the 7th Python in Science Conference*, Pasadena, CA, USA, 2008, pp. 11–15.
- [33] NVIDIA-AI-IOT. “Nanosam: A distilled segment anything model for real-time inference on nvidia jetson.” (2023), [Online]. Available: <https://github.com/NVIDIA-AI-IOT/nanosam> (visited on 10/02/2026).
- [34] NVIDIA. “Nvidia tensorrt documentation.” (2026), [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/> (visited on 10/02/2026).
- [35] A. Dosovitskiy, L. Beyer, A. Kolesnikov, *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations*, 2021.
- [36] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked autoencoders are scalable vision learners,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 16 000–16 009. doi: 10.1109/CVPR52688.2022.01553.
- [37] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” in *NIPS Deep Learning and Representation Learning Workshop*, arXiv:1503.02531, 2015.
- [38] C. Zhang, D. Han, Y. Qiao, J. U. Kim, S.-H. Bae, S. Lee, and C. S. Hong, “Faster segment anything: Towards lightweight SAM for mobile applications,” *arXiv preprint arXiv:2306.14289*, 2023.
- [39] K. Wu, J. Zhang, H. Peng, M. Liu, B. Xiao, J. Fu, and L. Yuan, “TinyViT: Fast pretraining distillation for small vision transformers,” in *European Conference on Computer Vision*, 2022, pp. 68–85.
- [40] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.
- [41] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *European Conference on Computer Vision*, 2014, pp. 740–755.
- [42] P. J. Huber, “Robust estimation of a location parameter,” *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.
- [43] J. McCormac, R. Clark, M. Bloesch, A. J. Davison, and S. Leutenegger, “Fusion++: Volumetric object-level slam,” in *2018 International Conference on 3D Vision (3DV)*, IEEE, 2018, pp. 32–41. doi: 10.1109/3DV.2018.00015.

Bibliography

- [44] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2961–2969. doi: 10.1109/ICCV.2017.322.
- [45] A. Radford, J. W. Kim, C. Hallacy, *et al.*, “Learning transferable visual models from natural language supervision,” in *Proceedings of the 38th International Conference on Machine Learning*, 2021, pp. 8748–8763.
- [46] P. Lewis, E. Perez, A. Piktus, *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [47] Chroma. “Chroma documentation.” (2026), [Online]. Available: <https://docs.trychroma.com/> (visited on 10/03/2026).
- [48] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [49] Y. Li, Y. Du, K. Zhou, J. Wang, W. X. Zhao, and J.-R. Wen, “Evaluating object hallucination in large vision-language models,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 292–305.
- [50] M. Xiong, Z. Hu, X. Lu, Y. Li, J. Fu, J. He, and B. Hooi, “Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs,” in *International Conference on Learning Representations*, 2024.
- [51] P. Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai, K. Chen, X. Liu, J. Wang, W. Ge, *et al.*, “Qwen2-VL: Enhancing vision-language model’s perception of the world at any resolution,” *arXiv preprint arXiv:2409.12191*, 2024.
- [52] S. Bai, K. Chen, X. Liu, *et al.*, “Qwen2.5-vl technical report,” *arXiv preprint arXiv:2502.13923*, 2025. arXiv: 2502.13923 [cs.CV].
- [53] Qwen Team, “Qwen3-VL technical report,” *arXiv preprint arXiv:2511.21631*, 2025.
- [54] Qwen Team. “Qwen3.5: Towards native multimodal agents.” (2026), [Online]. Available: <https://qwen.ai/blog?id=qwen3.5> (visited on 10/03/2026).
- [55] Artificial Analysis. “Qwen3.5 small models: Everything you need to know.” (2026), [Online]. Available: <https://artificialanalysis.ai/articles/qwen3-5-small-models> (visited on 10/03/2026).
- [56] NVIDIA. “Jetson agx orin for next-gen robotics.” (2026), [Online]. Available: <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/> (visited on 06/01/2026).
- [57] A. Schiel, “Online risk-annotated scene graphs for autonomous systems,” M.S. thesis, University of Stuttgart, Stuttgart, Germany, 2026.
- [58] NVIDIA. “Jetson thor: Advanced ai for physical robotics.” (2026), [Online]. Available: <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-thor/> (visited on 06/01/2026).

Bibliography

- [59] NVIDIA Jetson AI Lab. "Jetson ai lab models." (2026), [Online]. Available: <https://www.jetson-ai-lab.com/models/> (visited on 06/01/2026).

A. Example appendix chapter

Declaration of Compliance

I hereby declare to have written this work independently and to have respected in its preparation the relevant provisions, in particular those corresponding to the copyright protection of external materials. Whenever external materials (such as images, drawings, text passages) are used in this work, I declare that these materials are referenced accordingly (e. g. quote, source) and, whenever necessary, consent from the author to use such materials in my work has been obtained.

Signature:

A. Example appendix chapter

Stuttgart, on the 30 October 2026